Wyższa Szkoła Przedsiębiorczości i Administracji w Lublinie

Kierunek: Informatyka
Specjalność: Cyberbezpieczeństwo i informatyka śledcza
Tryb studiów: **stacjonarny**
Poziom studiów: **I stopnia**

Lokalny menedżer haseł z zaawansowanymi mechanizmami ochrony w architekturze zero-knowledge

(Local password manager with advanced protection mechanisms in a zero-knowledge architecture)

Autor: **Daryna Ovsiichuk**
Nr albumu: **20320**
Promotor: **mgr inż. Bartosz Sterniczuk**

Lublin 2026

Oświadczenie kierującego pracą.

Oświadczam, iż niniejsza praca została przygotowana pod moim kierunkiem i stwierdzam, że spełnia ona warunki do przedstawienia jej w postępowaniu o nadanie tytułu zawodowego.

…….. ……..

(data) (podpis)

Oświadczenie autora pracy

Świadomy odpowiedzialności prawnej oświadczam, iż niniejsza praca dyplomowa została napisana przeze mnie samodzielnie nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz.U. Nr 24, poz 83 z poźn. zm.) oraz dóbr osobistych chronionych prawem cywilnym dodatkowo nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Oświadczam również, że przedstawiona praca nie była wcześniej podmiotem procedur związanych z uzyskaniem tytułu zawodowego w wyższej uczelni.

Ponadto oświadczam, iż niniejsza wersja pracy jest identyczna z załączoną wersją elektroniczną.

…….. ……..

(data) (podpis)

STRESZCZENIE

Przedmiotem niniejszej pracy dyplomowej jest projekt oraz implementacja lokalnego menedżera haseł działającego w architekturze zerowej wiedzy (ang. *zero-knowledge*), w której całe szyfrowanie i deszyfrowanie odbywa się wyłącznie po stronie przeglądarki, a serwer przechowuje jedynie nieprzejrzysty szyfrotekst wraz z metadanymi i nigdy nie poznaje hasła głównego ani kluczy kryptograficznych. Operacja „logowania" sprowadza się do pobrania zaszyfrowanego pakietu danych oraz jego odszyfrowania w pamięci klienta; serwer pozostaje pasywnym magazynem bajtów, pozbawionym jakiejkolwiek logiki kryptograficznej.

Warstwę kryptograficzną zbudowano w oparciu o nowoczesne, sprawdzone prymitywy: twardą pamięciowo funkcję wyprowadzania klucza **Argon2id** (z kalibracją parametrów na urządzeniu oraz udokumentowanym wariantem zapasowym PBKDF2), szyfrowanie uwierzytelnione **AES-256-GCM** ze świeżym, losowym wektorem inicjującym dla każdej operacji oraz opakowanie klucza danych algorytmem **AES-KW** (RFC 3394) w modelu KEK/DEK. Klucze pozostają nieeksportowalne, a odszyfrowany skarbiec istnieje wyłącznie w ulotnym stanie aplikacji. Warstwę serwerową zrealizowano w technologii FastAPI z optymistyczną kontrolą współbieżności (ETag oraz nagłówek If-Match) i twardą walidacją schematu.

Rdzeniem pracy są tytułowe *zaawansowane mechanizmy ochrony*: fałszywy skarbiec otwierany pod przymusem (konstrukcja dwuslotowa o jednakowym czasie odblokowania), rotacja klucza danych, bramka siły hasła głównego oparta na entropii, automatyczna blokada po bezczynności, wykładnicze dławienie nieudanych prób oraz świadome fokusu czyszczenie schowka systemowego. W pracy przedstawiono analizę zagrożeń, przegląd istniejących rozwiązań, podstawy kryptograficzne, architekturę systemu, szczegółowy opis mechanizmów ochrony oraz bezpieczeństwa warstwy klienckiej, a także strategię testów wraz z wynikami ich wykonania (testy znanych odpowiedzi względem wektorów NIST/RFC, testy jednostkowe oraz scenariusze całościowe w przeglądarce). Wykazano, że możliwe jest zbudowanie wiarygodnego menedżera haseł, którego poprawność kryptograficzna jest weryfikowalna, a poufność danych nie zależy od zaufania do serwera.

Słowa kluczowe: menedżer haseł, architektura zerowej wiedzy, kryptografia po stronie klienta, Argon2id, AES-256-GCM, AES-KW, model KEK/DEK, entropia hasła, fałszywy skarbiec, bezpieczeństwo aplikacji webowych.

ABSTRACT

The subject of this thesis is the design and implementation of a local password manager operating within a *zero-knowledge* architecture, in which all encryption and decryption is performed exclusively on the browser side, while the server stores only opaque ciphertext together with its metadata and never learns the master password or any cryptographic keys. The act of "logging in" reduces to fetching the encrypted data package and decrypting it in the client's memory; the server remains a passive byte store, devoid of any cryptographic logic.

The cryptographic layer is built upon modern, well-established primitives: the memory-hard key derivation function **Argon2id** (with on-device parameter calibration and a documented PBKDF2 fallback), authenticated encryption with **AES-256-GCM** using a fresh random initialisation vector for every operation, and data-key wrapping with the **AES-KW** algorithm (RFC 3394) within a KEK/DEK model. Keys remain non-exportable, and the decrypted vault exists solely as ephemeral application state. The backend is implemented with the FastAPI framework, featuring optimistic concurrency control (ETag and the If-Match header) and strict schema validation.

The core of the thesis comprises the titular *advanced protection mechanisms*: a decoy vault opened under duress (a two-slot construction with a uniform unlock time), data-key rotation, an entropy-based master-password strength gate, automatic locking on inactivity, exponential throttling of failed attempts, and focus-aware clipboard clearing. The work presents a threat analysis, a review of existing solutions, the cryptographic foundations, the system architecture, a detailed description of the protection mechanisms and of front-end security, as well as the testing strategy together with the results of its execution (known-answer tests against NIST/RFC vectors, unit tests and browser-based end-to-end scenarios). It is demonstrated that it is feasible to build a trustworthy password manager whose cryptographic correctness is verifiable and whose data confidentiality does not depend on trusting the server.

Keywords: password manager, zero-knowledge, client-side cryptography, Argon2id, AES-256-GCM, AES-KW, KEK/DEK model, password entropy, decoy vault, web application security.

SPIS TREŚCI

Streszczenie

Abstract

Wstęp **4**

1 Wprowadzenie do problematyki i podstawowe pojęcia **7**
1.1 Problem zarządzania hasłami 7
1.2 Menedżery haseł i modele zaufania 8
1.3 Architektura zerowej wiedzy 9
1.4 Aparat pojęciowy pracy 10
1.5 Żelazne zasady projektowe 10

2 Przegląd istniejących rozwiązań i analiza zagrożeń **12**
2.1 Przegląd istniejących rozwiązań 12
2.2 Aktywa i profile napastników 13
2.3 Model zagrożeń 14
2.4 Wymagania bezpieczeństwa 17
2.5 Wymagania funkcjonalne 17
2.6 Pozycjonowanie własnego rozwiązania 18

3 Podstawy kryptograficzne **20**
3.1 Wymagania wobec warstwy kryptograficznej 20
3.2 Funkcje wyprowadzania klucza 20
3.2.1 Argon2id jako funkcja główna 21
3.2.2 PBKDF2 jako udokumentowany wariant zapasowy 22
3.3 Szyfrowanie uwierzytelnione AES-256-GCM 23
3.4 Opakowanie kluczy AES-KW 24
3.5 Model KEK/DEK 24
3.6 Kontrakt kryptograficzny i zakazy 26

4 Architektura systemu **27**
4.1 Przegląd architektury 27
4.2 Podział odpowiedzialności klient–serwer 28
4.3 Format pakietu danych 28
4.4 Serwer jako pasywny magazyn danych 30

4.5 Kontrola współbieżności 30
4.6 Nagłówki bezpieczeństwa i model dostępu 31
4.7 Przepływy tworzenia i odblokowania skarbca 32

5 Zaawansowane mechanizmy ochrony 34
5.1 Fałszywy skarbiec „pod przymusem” 34
5.2 Rotacja klucza danych 36
5.3 Bramka siły hasła głównego 37
5.4 Auto-blokada po bezczynności 38
5.5 Dławienie nieudanych prób 39
5.6 Świadome fokusu czyszczenie schowka 40
5.7 Kalibracja funkcji Argon2id 41
5.8 Podsumowanie mechanizmów 41

6 Bezpieczeństwo warstwy klienckiej 43
6.1 Warstwa kliencka jako pierwsza linia obrony 43
6.2 Ścisła polityka bezpieczeństwa treści 43
6.3 Renderowanie treści jako tekstu 44
6.4 Nieeksportowalność kluczy 45
6.5 Brak sekretów w pamięci trwałej 45
6.6 Granica błędów jako mechanizm bezpieczeństwa 46
6.7 Dostępność jako element jakości i bezpieczeństwa 46

7 Implementacja i jakość 47
7.1 Stos technologiczny i jego uzasadnienie 47
7.2 Kluczowe komponenty 48
7.3 Strategia testów 48
7.3.1 Testy jednostkowe oraz testy odwracalności 49
7.3.2 Testy znanych odpowiedzi 49
7.3.3 Testy całościowe 50
7.4 Wyniki wykonania testów 50
7.4.1 Wyniki testów warstwy klienckiej 51
7.4.2 Wyniki testów warstwy serwerowej 52
7.4.3 Pokrycie kodu 54
7.4.4 Wyniki testów całościowych 56
7.4.5 Podsumowanie wyników 59
7.5 Ciągła integracja i audyt łańcucha dostaw 59

8 Weryfikacja, ograniczenia i kierunki rozwoju 60
8.1 Weryfikacja poprawności i wydajności 60
8.2 Uczciwe ograniczenia 61
8.3 Kierunki rozwoju 62

Zakończenie **64**

Bibliografia **66**

Spis rysunków **69**

Spis tabel **71**

Spis listingów **72**

Załączniki **73**

A Słownik terminów i skrótów **74**

B Przykładowa struktura pakietu danych **76**

C Struktura katalogów projektu **77**

D Zrzuty ekranu z działania aplikacji **78**

WSTĘP

Hasło pozostaje, mimo wieloletnich zapowiedzi jego zmierzchu, podstawowym mechanizmem uwierzytelniania w sieci [5]. Przeciętny użytkownik posiada dziś dziesiątki, a często setki kont w usługach internetowych, z których każde wymaga osobnych poświadczeń. Ludzka pamięć nie jest w stanie sprostać temu obciążeniu, czego naturalną konsekwencją są praktyki obniżające bezpieczeństwo: stosowanie haseł krótkich i łatwych do odgadnięcia, a przede wszystkim wielokrotne używanie tego samego hasła w wielu serwisach. To ostatnie zjawisko czyni użytkownika podatnym na ataki polegające na masowym podstawianiu wykradzionych poświadczeń (ang. *credential stuffing*), w których dane logowania przejęte z jednej, słabiej zabezpieczonej usługi służą do opanowania kont w usługach zupełnie niepowiązanych. Odpowiedzią na ten problem są *menedżery haseł* [24], programy przechowujące poświadczenia w zaszyfrowanej postaci i udostępniające je użytkownikowi po podaniu pojedynczego *hasła głównego*.

Menedżery haseł, choć rozwiązują problem zapamiętywania, jednocześnie tworzą nowy: gromadzą w jednym miejscu wszystkie sekrety użytkownika, stając się celem o bardzo wysokiej wartości. Szczególnego znaczenia nabiera tu kwestia zaufania do dostawcy usługi. W popularnym modelu chmurowym zaszyfrowany skarbiec przechowywany jest na serwerach dostawcy, a bezpieczeństwo danych opiera się na założeniu, że dostawca prawidłowo zaprojektował i wdrożył kryptografię oraz że jego infrastruktura nie zostanie skompromitowana. Założenie to bywa zawodne, o czym świadczą udokumentowane incydenty naruszenia bezpieczeństwa u komercyjnych dostawców menedżerów haseł, w wyniku których w ręce atakujących trafiały kopie zaszyfrowanych skarbców wraz z metadanymi [15]. Po takim wycieku jedyną barierą chroniącą dane użytkownika pozostaje odporność hasła głównego na atak siłowy prowadzony w trybie offline.

Architekturą, która minimalizuje wymagane zaufanie do serwera, jest model zerowej wiedzy (ang. *zero-knowledge*). W modelu tym całość operacji kryptograficznych, wyprowadzenie klucza z hasła, szyfrowanie i deszyfrowanie, odbywa się wyłącznie po stronie klienta, na urządzeniu użytkownika. Serwer nigdy nie otrzymuje ani hasła głównego, ani klucza, ani danych w postaci jawnej; przechowuje wyłącznie *nieprzejrzysty szyfrotekst*. Dzięki temu kompromitacja serwera nie prowadzi automatycznie do ujawnienia treści skarbca, a poufność danych przestaje zależeć od zaufania do operatora usługi. To właśnie ten model stanowi fundament projektowy rozwiązania opisanego w niniejszej pracy.

Aktualność podjętej tematyki nie budzi wątpliwości. Skala wycieków poświadczeń rośnie z roku na rok, co dokumentują zarówno coroczne raporty branżowe, jak i publicznie dostępne zbiory naruszeń [7, 12, 29], a ponowne wykorzystywanie haseł pozostaje jedną

z najczęstszych przyczyn skutecznych włamań [11, 24, 29]. Jednocześnie rozwiązania komercyjne, choć dojrzałe, są zamknięte: użytkownik nie ma możliwości zweryfikowania, w jaki sposób jego dane są chronione. Stwarza to przestrzeń dla rozwiązania w pełni przejrzystego, którego kryptografia jest możliwa do prześledzenia na poziomie kodu źródłowego, a jej poprawność, weryfikowalna za pomocą testów. Podstawowe pojęcia kryptograficzne (KDF, AEAD, model KEK/DEK) zdefiniowano w rozdziale 1.

Cel pracy: Głównym celem pracy jest zaprojektowanie, zaimplementowanie oraz weryfikacja lokalnego menedżera haseł działającego w architekturze zerowej wiedzy, w którym serwer fizycznie nie jest w stanie poznać hasła głównego ani kluczy użytkownika, wyposażonego ponadto w zestaw zaawansowanych mechanizmów ochrony wykraczających poza standard typowego menedżera haseł. Cel główny zrealizowano poprzez następujące cele szczegółowe:

- analizę zagrożeń dla menedżerów haseł oraz przegląd istniejących rozwiązań,
- dobór i uzasadnienie współczesnych prymitywów kryptograficznych (KDF, AEAD, opakowanie kluczy) oraz modelu klucza,
- zaprojektowanie architektury zerowej wiedzy wraz z formatem przechowywanych danych i pasywnym serwerem pozbawionym logiki kryptograficznej,
- zaprojektowanie i implementację zaawansowanych mechanizmów ochrony, ze szczególnym uwzględnieniem fałszywego skarbca otwieranego pod przymusem,
- zabezpieczenie warstwy klienckiej (ang. *front-end*) jako pierwszej linii obrony w architekturze webowej,
- weryfikację poprawności rozwiązania za pomocą testów, w tym testów znanych odpowiedzi względem oficjalnych wektorów kryptograficznych.

Założenia projektowe: U podstaw projektu przyjęto zestaw założeń, które wyznaczają kształt rozwiązania i stanowią kryterium oceny stopnia realizacji celów. Założono, że lokalny menedżer haseł w architekturze zerowej wiedzy da się zbudować tak, aby poufność i integralność danych nie zależały od zaufania do serwera, a poprawność warstwy kryptograficznej podlegała weryfikacji automatycznej. Przyjęto ponadto, że rozwiązanie takie można wyposażyć w zaawansowane mechanizmy ochrony przy jawnym nazwaniu ich ograniczeń. Stopień spełnienia tych założeń oceniono w zakończeniu pracy.

Zakres i ograniczenia pracy: Praca obejmuje pełen cykl wytwórczy prototypu: analizę, projekt, implementację oraz weryfikację. Świadomie ograniczono zasięg rozwiązania do pojedynczego użytkownika i pojedynczego skarbca, kładąc nacisk na **poprawność kryptografii, architekturę i bezpieczeństwo**, a nie na liczbę funkcji. Ograniczenia rozwiązania potraktowano jako pełnoprawny przedmiot analizy i omówiono w pracy w sposób jawny.

Struktura pracy: Praca składa się z ośmiu rozdziałów: od wprowadzenia, przeglądu istniejących rozwiązań wraz z modelem zagrożeń oraz podstaw kryptograficznych (rozdziały 1–3), przez architekturę systemu (rozdział 4) i rdzeń pracy, zaawansowane mechanizmy ochrony (rozdział 5), po bezpieczeństwo warstwy klienckiej, implementację oraz weryfikację wraz z kierunkami rozwoju (rozdziały 6–8). Pracę zamyka zakończenie podsumowujące stopień realizacji przyjętych celów i założeń projektowych.

Rozdział 1

WPROWADZENIE DO PROBLEMATYKI I PODSTAWOWE POJĘCIA

Rozdział niniejszy wprowadza w problematykę zarządzania hasłami oraz ustanawia aparat pojęciowy wykorzystywany w dalszej części pracy. Najpierw scharakteryzowano problem haseł w skali współczesnej sieci oraz menedżery haseł jako klasę rozwiązań, zwracając uwagę na różnice w modelach zaufania pomiędzy rozwiązaniami chmurowymi a lokalnymi. Następnie przedstawiono architekturę zerowej wiedzy jako odpowiedź na ograniczenia modelu chmurowego oraz przywołano aparat pojęciowy, którym posługuje się część analityczna pracy. Rozdział zamyka zestaw żelaznych zasad projektowych, stanowiących nieprzekraczalne ramy całego przedsięwzięcia.

1.1 Problem zarządzania hasłami

Uwierzytelnianie oparte na haśle pozostaje dominującym mechanizmem kontroli dostępu w usługach internetowych [5]. Jego trwałość wynika z prostoty wdrożenia i uniwersalności: hasło nie wymaga żadnej dodatkowej infrastruktury po stronie użytkownika i działa w każdym środowisku. Ta sama prostota jest jednak źródłem zasadniczego napięcia. Bezpieczne hasło powinno być długie, losowe i unikalne dla każdej usługi [11], tymczasem hasło łatwe do zapamiętania jest z natury krótkie, przewidywalne i chętnie powtarzane. W sytuacji, gdy pojedynczy użytkownik utrzymuje dziesiątki lub setki kont, pogodzenie tych wymagań bez narzędzia wspomagającego jest praktycznie niemożliwe.

Najpoważniejszą konsekwencją tego napięcia jest *ponowne wykorzystywanie haseł* (ang. *password reuse*), zjawisko szeroko udokumentowane zarówno w wytycznych normalizacyjnych, jak i w badaniach zachowań użytkowników [11, 24]. Gdy to samo hasło chroni wiele kont, wyciek poświadczeń z jednej, słabiej zabezpieczonej usługi otwiera atakującemu drogę do wszystkich pozostałych. Zautomatyzowane ataki tej klasy, określane jako podstawianie wykradzionych poświadczeń (ang. *credential stuffing*) [21], polegają na masowym testowaniu par login–hasło, pozyskanych z wcześniejszych wycieków, wobec niepowiązanych serwisów. Skuteczność takich ataków jest bezpośrednią pochodną skali zjawiska powtarzania haseł [29] i nie da się jej wyeliminować po stronie atakowanej usługi, leży ona w nawykach użytkownika.

Rozwiązaniem, które rozcina ten węzeł, jest *menedżer haseł*. Narzędzie to przejmuje od użytkownika obowiązek pamiętania poświadczeń: przechowuje je w zaszyfrowanej postaci,

a do ich odblokowania wymaga pojedynczego *hasła głównego*. Dzięki temu użytkownik może stosować dla każdej usługi osobne, długie i losowe hasło, zapamiętując w zamian tylko jeden silny sekret. Menedżer haseł nie usuwa jednak problemu, lecz przenosi go: zamiast wielu słabych punktów powstaje jeden punkt krytyczny, którego kompromitacja oznacza ujawnienie wszystkich sekretów naraz [16]. Bezpieczeństwo całego rozwiązania zależy więc od siły hasła głównego oraz od poprawności mechanizmów chroniących skarbiec.

1.2 Menedżery haseł i modele zaufania

Menedżery haseł różnią się przede wszystkim miejscem przechowywania skarbca oraz wynikającym z niego *modelem zaufania*, czyli zbiorem podmiotów, którym użytkownik musi ufać, by jego dane pozostały poufne. Typologia ta jest w literaturze przedmiotu ugruntowana i stanowi punkt wyjścia analiz bezpieczeństwa menedżerów haseł [5, 16]. Rozwiązania *lokalne* przechowują zaszyfrowany skarbiec wyłącznie na urządzeniu użytkownika, ograniczając zaufanie do samego urządzenia i oprogramowania; przykładem tej klasy jest KeePass [14]. Rozwiązania *chmurowe* synchronizują skarbiec poprzez serwery dostawcy, dodając do modelu zaufania operatora usługi oraz jego infrastrukturę, co dostawcy dokumentują we własnych opracowaniach dotyczących projektu zabezpieczeń [1, 4].

Kluczowe jest rozróżnienie, *co dokładnie* powierza się serwerowi w modelu chmurowym. W podejściu naiwnym serwer mógłby przechowywać dane w postaci jawnej lub dysponować kluczem deszyfrującym, wówczas kompromitacja serwera oznaczałaby natychmiastowe ujawnienie wszystkich skarbców. W dojrzałym podejściu serwer otrzymuje wyłącznie szyfrotekst, a klucz nigdy go nie opuszcza urządzenia klienta. Tę drugą zasadę formalizuje architektura zerowej wiedzy, omówiona w sekcji 1.3. Tabela 1.1 zestawia rozważane modele zaufania.

Tab. 1.1: Modele przechowywania skarbca a wymagane zaufanie

Model	**Komu trzeba ufać**	**Skutek kompromitacji serwera**
Chmurowy naiwny	Dostawcy, jego infrastrukturze i personelowi	Natychmiastowe ujawnienie danych jawnych lub kluczy
Chmurowy z zerową wiedzą	Poprawności klienta; serwer poza zaufaniem	Ujawnienie wyłącznie szyfrotekstu, bariera: siła hasła głównego
Lokalny (niniejsza praca)	Urządzeniu i oprogramowaniu klienta	Brak, serwer nie przechowuje danych w postaci użytecznej

Źródło: opracowanie własne

Rozwiązanie opisane w niniejszej pracy jest menedżerem *lokalnym* zbudowanym zgodnie z zasadą zerowej wiedzy: nawet komponent serwerowy, choć obecny, jest celowo pozbawiony wiedzy o treści skarbca. Takie połączenie minimalizuje model zaufania, zachowując jednocześnie architekturę klient–serwer, która w przyszłości umożliwia rozbudowę o synchronizację bez naruszania zasady zerowej wiedzy.

1.3 Architektura zerowej wiedzy

Architektura zerowej wiedzy (ang. *zero-knowledge*) opiera się na zasadzie, że serwer nigdy nie wchodzi w posiadanie informacji pozwalających odtworzyć treść chronionych danych; w tym modelu działają dojrzałe rozwiązania komercyjne [1, 4]. W kontekście menedżera haseł oznacza to, że całość operacji kryptograficznych, wyprowadzenie klucza z hasła głównego, szyfrowanie oraz deszyfrowanie skarbca, wykonywana jest wyłącznie po stronie klienta, na urządzeniu użytkownika. Serwer otrzymuje i zwraca jedynie *nieprzejrzysty szyfrotekst* wraz z metadanymi niezbędnymi do jego odtworzenia (solą, parametrami funkcji wyprowadzania klucza, wektorem inicjującym oraz numerem wersji), które z jego punktu widzenia są ciągiem bajtów pozbawionym znaczenia.

Konsekwencją tej zasady jest rola serwera jako pasywnego magazynu bajtów. Operacja, którą użytkownik postrzega jako „logowanie", nie jest w istocie żadną wymianą poświadczeń z serwerem: polega ona na pobraniu zaszyfrowanego pakietu danych i podjęciu próby jego odszyfrowania w pamięci przeglądarki. Powodzenie tej próby jest jedynym dowodem znajomości prawidłowego hasła, dowodem, który nigdy nie opuszcza urządzenia klienta. Z tego wynika fundamentalna własność modelu: serwer jest *fizycznie niezdolny* do poznania hasła głównego, ponieważ hasło to nigdy nie jest mu przesyłane w żadnej postaci, również

jako skrót.

Praktycznym następstwem architektury zerowej wiedzy jest przesunięcie ciężaru bezpieczeństwa na warstwę klienta. Skoro serwer nie zna danych, nie może też ich chronić; cała kryptografia oraz powierzchnia ataku przenosi się do przeglądarki. Czyni to bezpieczeństwo warstwy klienckiej zagadnieniem pierwszorzędnym, wątek ten rozwinięto w rozdziale 6. Jednocześnie model ten radykalnie ogranicza skutki kompromitacji serwera: atakujący, który przejmie pełną kontrolę nad bazą danych, uzyskuje wyłącznie zbiór szyfrotekstów, których złamanie wymaga ataku siłowego na hasło główne, kosztownego dzięki właściwie dobranej funkcji wyprowadzania klucza.

1.4 Aparat pojęciowy pracy

Dalszy wywód posługuje się kilkoma pojęciami kryptograficznymi, przywołanymi tu wyłącznie w zakresie niezbędnym do lektury części analitycznej. *Funkcja wyprowadzania klucza* (ang. *Key Derivation Function*, KDF) przekształca hasło o niskiej entropii w klucz kryptograficzny, czyniąc to celowo kosztownie i z udziałem jawnej, losowej *soli* (ang. *salt*), która sprawia, że identyczne hasła dwóch użytkowników dają różne klucze. *Wektor inicjujący* (ang. *Initialization Vector*, IV) to losowa, również jawna wartość gwarantująca, że dwukrotne zaszyfrowanie tej samej treści tym samym kluczem da różne szyfroteksty. *Szyfrowanie uwierzytelnione* (ang. *Authenticated Encryption with Associated Data*, AEAD) łączy poufność z integralnością: obok ukrycia treści generuje znacznik pozwalający wykryć każdą modyfikację szyfrotekstu. *Model KEK/DEK* rozdziela klucz wyprowadzony z hasła, czyli klucz opakowujący (ang. *Key Encryption Key*), od losowego klucza danych (ang. *Data Encryption Key*) szyfrującego skarbiec, dzięki czemu zmiana hasła nie wymaga ponownego szyfrowania całości. Wreszcie *entropia*, mierzona w bitach, wyraża nieprzewidywalność sekretu i w architekturze zerowej wiedzy stanowi ostateczną barierę chroniącą skarbiec po wycieku szyfrotekstu [9, 11, 13].

Podział ról pomiędzy częściami pracy jest przy tym ścisły i celowo bezkolizyjny. Niniejszy podrozdział pojęcia te wyłącznie **nazywa**, aby rozdział 2 mógł się nimi posługiwać. **Uzasadnienie wyboru** konkretnych algorytmów, które je realizują, wraz z analizą odrzuconych alternatyw, zawiera rozdział 3; praktyczne wykorzystanie entropii w bramce siły hasła głównego omówiono w rozdziale 5, a zwięzłe definicje wszystkich terminów i skrótów zestawiono w dodatku A.

1.5 Żelazne zasady projektowe

Realizacja oparta jest na zbiorze nieprzekraczalnych zasad, które wyznaczają granice wszystkich późniejszych decyzji projektowych i implementacyjnych. Zasady te wynikają wprost z przyjętego modelu zerowej wiedzy i mają charakter wiążący.

1. **Hasło główne nigdy nie opuszcza przeglądarki.** Komponent serwerowy musi być fizycznie niezdolny do jego odebrania; zakazane jest przesyłanie hasła ani jego skrótu w roli poświadczenia.
2. **Serwer przechowuje wyłącznie nieprzejrzysty szyfrotekst i metadane** (sól, parametry KDF, wektor inicjujący, numer wersji). Po stronie serwera nie istnieje żadna logika kryptograficzna.
3. **Klucze pozostają nieeksportowalne.** Materiał klucza jest utrzymywany jako obiekt nieudostępniający surowych bajtów, a bufory zawierające surowy wynik funkcji KDF są zerowane bezpośrednio po imporcie.
4. **Żadnych sekretów w pamięci trwałej przeglądarki.** Ani klucz, ani odszyfrowany skarbiec nie trafiają do magazynów typu `localStorage` czy `sessionStorage`; odszyfrowane dane istnieją wyłącznie w ulotnym stanie aplikacji.
5. **Podatność typu XSS oznacza całkowitą kompromitację** [23]. Skoro odszyfrowany skarbiec żyje w pamięci klienta, wykonanie obcego skryptu daje do niego dostęp; dlatego treść wpisów renderowana jest zawsze jako tekst, nigdy jako kod HTML.

Zasady te przewijają się przez całą pracę i stanowią kryterium oceny każdej podejmowanej decyzji. Ich techniczne konsekwencje dla warstwy kryptograficznej, architektury oraz warstwy klienckiej omówiono odpowiednio w rozdziałach 3, 4 oraz 6.

Rozdział 2

PRZEGLĄD ISTNIEJĄCYCH ROZWIĄZAŃ I ANALIZA ZAGROŻEŃ

Rozdział ten stanowi część analityczną pracy. Jego celem jest osadzenie projektu w krajobrazie istniejących rozwiązań oraz w realnym kontekście zagrożeń, a następnie przełożenie wniosków z tej analizy na konkretne wymagania, które projektowany system musi spełnić. Wywód prowadzony jest zatem od stanu istniejącego ku wymaganiom. Najpierw dokonano przeglądu i porównania reprezentatywnych menedżerów haseł, rozpoznając rozwiązania stosowane w praktyce oraz ich słabości. Następnie zdefiniowano aktywa podlegające ochronie i profile napastników, po czym zbudowano model zagrożeń obejmujący najistotniejsze wektory ataku na menedżer haseł. Dopiero z tak przygotowanej analizy wyprowadzono wymagania bezpieczeństwa oraz wymagania funkcjonalne. Rozdział zamyka pozycjonowanie rozwiązania opisanego w niniejszej pracy na tle omówionych menedżerów.

2.1 Przegląd istniejących rozwiązań

Analizę otwiera przegląd czterech reprezentatywnych menedżerów haseł, dobranych tak, by pokryć główne modele architektoniczne obecne na rynku. Rozpoznanie rozwiązań już stosowanych, a zwłaszcza ich udokumentowanych słabości, stanowi naturalny punkt wyjścia zarówno dla modelu zagrożeń, jak i dla wymagań formułowanych w dalszej części rozdziału; na tle tego przeglądu spozycjonowano następnie projekt własny (podrozdział 2.6).

Bitwarden: Rozwiązanie chmurowe o otwartym kodzie źródłowym, działające w modelu zerowej wiedzy [4]: szyfrowanie i deszyfrowanie odbywają się po stronie klienta, a serwer przechowuje zaszyfrowany skarbiec. Bitwarden oferuje synchronizację wielu urządzeń, aplikacje na liczne platformy oraz możliwość samodzielnego hostowania serwera. Jego architektura jest koncepcyjnie najbliższa rozwiązaniu opisanemu w niniejszej pracy, różni się jednak skalą, dojrzałością oraz nastawieniem na pracę w chmurze.

KeePass: Klasyczny menedżer w pełni lokalny, przechowujący skarbiec w pojedynczym zaszyfrowanym pliku bazy danych [14]. KeePass nie posiada wbudowanej synchronizacji chmurowej, użytkownik samodzielnie zarządza plikiem i jego ewentualną synchronizacją. Model zaufania jest tu minimalny i ograniczony do urządzenia, co czyni KeePass bliskim filozofii niniejszej pracy w warstwie lokalności, przy odmiennym, plikowym modelu dystrybucji i braku komponentu serwerowego.

1Password: Dojrzałe rozwiązanie komercyjne o zamkniętym kodzie, działające w modelu chmurowym zerowej wiedzy. Wyróżnia je zastosowanie dodatkowego, generowanego losowo *klucza sekretnego* łączonego z hasłem głównym, co podnosi odporność na atak siłowy [1]. Jako produkt komercyjny pozostaje jednak nieprzejrzysty dla użytkownika, który musi przyjąć deklarowane właściwości bezpieczeństwa na wiarę.

LastPass: Popularne rozwiązanie chmurowe, które posłużyło jako instruktywny przykład ryzyk modelu chmurowego. Ujawnione incydenty naruszenia bezpieczeństwa, w wyniku których w ręce napastników trafiły kopie zaszyfrowanych skarbców wraz z częścią metadanych [15], unaoczniły, że bezpieczeństwo użytkownika sprowadza się wówczas do odporności jego hasła głównego na atak offline. Przypadek ten dobitnie uzasadnia nacisk niniejszej pracy na silną funkcję wyprowadzania klucza oraz bramkę siły hasła.

Tabela 2.1 zestawia omówione rozwiązania według wybranych kryteriów.

Tab. 2.1: Porównanie wybranych menedżerów haseł

Cecha	**Bitwarden**	**KeePass**	**1Password**	**Niniejsza praca**
Model	chmurowy	lokalny	chmurowy	lokalny, zerowa wiedza
Otwarty kod	tak	tak	nie	tak
Zerowa wiedza	tak	nie dotyczy	tak	tak
Synchronizacja	tak	ręczna	tak	poza zakresem
Fałszywy skarbiec	nie	nie	nie	tak

Źródło: opracowanie własne na podstawie [1, 4, 14]

2.2 Aktywa i profile napastników

Punktem wyjścia analizy zagrożeń jest precyzyjne wskazanie *aktywów*, czyli zasobów, których ochrona jest celem systemu. W rozważanym menedżerze haseł aktywem o najwyższej wartości jest **hasło główne**, ponieważ z niego wyprowadzane są wszystkie klucze, a jego ujawnienie oznacza utratę całego skarbca. Drugim aktywem jest **klucz danych** (DEK) oraz, w dalszej kolejności, **odszyfrowana treść skarbca** istniejąca w pamięci odblokowanej sesji. Aktywem o niższej wrażliwości, lecz wciąż istotnym, są **metadane**, choć nie zawierają sekretów, mogą zdradzać liczbę wpisów, częstotliwość zmian czy obecność dodatkowych struktur w pakiecie danych.

Analizę uzupełnia charakterystyka napastników, których model systemu bierze pod uwagę; przyjęte profile odpowiadają klasom zagrożeń rozpoznanym w analizach bezpieczeństwa menedżerów haseł [16, 23]. *Napastnik sieciowy* przechwytuje lub modyfikuje ruch między przeglądarką a serwerem. *Napastnik serwerowy* przejmuje kontrolę nad komponentem serwerowym i jego bazą danych. To scenariusz, wobec którego architektura zerowej wiedzy wykazuje swoją największą wartość. *Napastnik przeglądarkowy* potrafi doprowadzić do wykonania obcego kodu w kontekście aplikacji, na przykład poprzez atak typu XSS. *Napastnik fizyczny* uzyskuje dostęp do urządzenia użytkownika lub kopii pakietu danych, a w skrajnym przypadku *napastnik wymuszający* zmusza użytkownika do podania hasła. Każdy z tych profili odpowiada innemu zestawowi mechanizmów obronnych, omówionych poniżej.

2.3 Model zagrożeń

Model zagrożeń systematyzuje najważniejsze wektory ataku oraz przyporządkowuje im mechanizmy obronne wbudowane w architekturę rozwiązania. Tabela 2.2 przedstawia jego skrótowe zestawienie, natomiast dalsza część sekcji omawia poszczególne zagrożenia szczegółowo.

Tab. 2.2: Model zagrożeń projektowanego menedżera haseł

Zagrożenie	**Mechanizm obronny**
Kradzież serwera lub bazy danych	Architektura zerowej wiedzy, serwer przechowuje wyłącznie szyfrotekst
Atak siłowy na hasło główne (offline)	Twarda pamięciowo funkcja KDF Argon2id oraz bramka siły hasła głównego
Podmiana lub uszkodzenie szyfrotekstu	Szyfrowanie uwierzytelnione AES-256-GCM (wykrycie naruszenia znacznikiem)
Wstrzyknięcie skryptu (XSS)	Ścisły CSP, renderowanie treści jako tekstu, brak skryptów osadzonych w treści strony
Wyciek klucza z pamięci JavaScript	Klucze nieeksportowalne, brak sekretów w pamięci trwałej
Wymuszenie podania hasła	Fałszywy skarbiec otwierany pod przymusem (z udokumentowanym ograniczeniem)
Ciche nadpisanie zmian z innej karty	Optymistyczna kontrola współbieżności (ETag oraz If-Match)
Podatność w zależności zewnętrznej	Bramki audytu `npm audit` oraz `pip-audit` w procesie CI

Źródło: opracowanie własne na podstawie polityki bezpieczeństwa projektu

Kradzież serwera lub bazy danych: Jest to scenariusz, wobec którego klasyczne menedżery chmurowe są najbardziej narażone, a architektura zerowej wiedzy wykazuje największą odporność. Ponieważ serwer przechowuje wyłącznie nieprzejrzysty szyfrotekst, a klucz nigdy nie opuszcza urządzenia klienta, przejęcie pełnej kopii bazy danych nie daje napastnikowi dostępu do treści skarbca, co odróżnia ten model od udokumentowanych wycieków u dostawców chmurowych [15]. Pozostaje mu wyłącznie atak siłowy na hasło główne prowadzony w trybie offline, którego koszt jest świadomie podnoszony przez funkcję wyprowadzania klucza.

Atak siłowy na hasło główne: Po wycieku szyfrotekstu jedyną barierą staje się odporność hasła głównego na zgadywanie. Obrona jest tu dwuwarstwowa. Po pierwsze, funkcja **Argon2id** jest twarda pamięciowo, każda próba zgadnięcia wymaga zaangażowania znacznej ilości pamięci, co drastycznie podnosi koszt ataku zrównoleglonego na procesorach graficznych i układach specjalizowanych [2, 3]. Po drugie, **bramka siły hasła głównego** uniemożliwia utworzenie skarbca chronionego hasłem o zbyt niskiej entropii, czyli takim, którego żaden koszt KDF nie zdoła obronić. Oba mechanizmy są komplementarne: KDF

podnosi koszt jednej próby, bramka zaś ogranicza minimalną liczbę prób koniecznych do sukcesu.

Podmiana lub uszkodzenie szyfrotekstu: Napastnik kontrolujący serwer mógłby próbować zmodyfikować przechowywany szyfrotekst, licząc na zakłócenie działania klienta lub podstawienie spreparowanych danych. Zastosowanie szyfrowania uwierzytelnionego AES-256-GCM [6] sprawia, że każda, choćby jednobajtowa, modyfikacja szyfrotekstu jest wykrywana podczas deszyfrowania: weryfikacja znacznika uwierzytelniającego kończy się niepowodzeniem, a klient odmawia przyjęcia danych zamiast operować na przekłamanej treści.

Wstrzyknięcie skryptu (XSS): W architekturze zerowej wiedzy to właśnie atak XSS stanowi najpoważniejsze zagrożenie [23], ponieważ wykonanie obcego skryptu w kontekście odblokowanej sesji daje napastnikowi dostęp do odszyfrowanego skarbca w pamięci. Z tego powodu zagrożenie to jest traktowane jako potencjalna pełna kompromitacja, a obrona ma charakter warstwowy: ścisła polityka bezpieczeństwa treści (CSP) blokująca skrypty osadzone w treści strony oraz skrypty zdalne, bezwzględne renderowanie treści wpisów jako tekstu oraz rezygnacja z mechanizmów osadzających surowy kod HTML. Zagadnienie to rozwinięto w rozdziale 6.

Wyciek klucza z pamięci JavaScript: Nawet bez pełnej kompromitacji środowiska wykonawczego pożądane jest ograniczenie możliwości eksfiltracji surowych bajtów klucza. Służy temu przechowywanie kluczy jako obiektów nieeksportowalnych, których materiał nie jest dostępny dla kodu JavaScript [18], oraz konsekwentne nieumieszczanie jakichkolwiek sekretów w pamięci trwałej przeglądarki. Mechanizmy te nie eliminują skutków pełnej kompromitacji, lecz podnoszą poprzeczkę napastnikowi i ograniczają trwałość ewentualnego wycieku.

Wymuszenie podania hasła: Scenariusz, w którym napastnik zmusza użytkownika do otwarcia skarbca, wykracza poza klasyczną kryptografię. Odpowiedzią jest **fałszywy skarbiec**, wiarygodny, lecz niegroźny skarbiec, który użytkownik może okazać pod naciskiem. Mechanizm ten, będący rdzeniem pracy, omówiono szczegółowo w rozdziale 5 wraz z jego uczciwie nazwanym ograniczeniem: zapewnia on ochronę przed wymuszeniem, lecz nie kryptograficzną zaprzeczalność.

Ciche nadpisanie zmian oraz podatne zależności: Dwa pozostałe zagrożenia mają charakter raczej integralnościowy i procesowy niż ściśle kryptograficzny. Optymistyczna kontrola współbieżności chroni przed cichą utratą zmian wprowadzonych równolegle z innej karty lub urządzenia, natomiast bramki audytu zależności w procesie ciągłej integracji ograniczają ryzyko wprowadzenia do projektu znanej podatności poprzez łańcuch dostaw oprogramowania [23].

2.4 Wymagania bezpieczeństwa

Z modelu zagrożeń wyprowadzono zbiór wymagań bezpieczeństwa o charakterze niefunkcjonalnym, które wyznaczają nieprzekraczalne właściwości systemu. Zestawiono je w tabeli 2.3.

Tab. 2.3: Wymagania bezpieczeństwa projektowanego systemu

Nr	Wymaganie bezpieczeństwa
WB-1	Hasło główne nigdy nie jest przesyłane do serwera w żadnej postaci
WB-2	Serwer przechowuje wyłącznie nieprzejrzysty szyfrotekst oraz metadane
WB-3	Wyprowadzanie klucza realizuje twarda pamięciowo funkcja KDF z udokumentowanym wariantem zapasowym
WB-4	Szyfrowanie skarbca zapewnia poufność oraz integralność (AEAD)
WB-5	Klucze kryptograficzne pozostają nieeksportowalne
WB-6	Żaden sekret nie jest zapisywany w pamięci trwałej przeglądarki
WB-7	Treść wpisów jest renderowana wyłącznie jako tekst, nigdy jako HTML
WB-8	Aplikacja egzekwuje ścisłą politykę bezpieczeństwa treści (CSP)
WB-9	Odblokowany skarbiec jest automatycznie blokowany po okresie bezczynności
WB-10	Nieudane próby odblokowania podlegają narastającemu opóźnieniu
WB-11	System odrzuca nieaktualne zapisy chroniąc przed cichym nadpisaniem zmian
WB-12	Poprawność prymitywów kryptograficznych jest weryfikowana automatycznie

Źródło: opracowanie własne

Wymagania te mają charakter wiążący i stanowią kryterium oceny rozwiązania. Część z nich, WB-1, WB-2, WB-12, podlega weryfikacji automatycznej w testach i scenariuszach całościowych, co opisano w rozdziale 7.

2.5 Wymagania funkcjonalne

Obok wymagań bezpieczeństwa zdefiniowano wymagania funkcjonalne określające zakres funkcji dostępnych dla użytkownika. Zestawiono je w tabeli 2.4.

Tab. 2.4: Wymagania funkcjonalne projektowanego systemu

Nr	Wymaganie funkcjonalne
WF-1	Utworzenie skarbca chronionego hasłem głównym oraz jego odblokowanie
WF-2	Dodawanie, edycja i usuwanie wpisów (tytuł, login, hasło, URL, notatki)
WF-3	Organizacja wpisów za pomocą tagów oraz oznaczania jako ulubione
WF-4	Generowanie haseł losowych oraz fraz hasłowych (ang. *diceware*) z miernikiem entropii
WF-5	Audyt zdrowia haseł (słabe, zbyt krótkie, powtórzone, przeterminowane)
WF-6	Lokalne wyliczanie kodów jednorazowych TOTP [20] na podstawie zapisanego sekretu
WF-7	Zmiana hasła głównego bez ponownego szyfrowania całego skarbca
WF-8	Eksport oraz import zaszyfrowanej kopii zapasowej
WF-9	Kosz z miękkim usuwaniem oraz możliwością przywrócenia wpisu
WF-10	Kopiowanie poświadczeń do schowka z automatycznym czyszczeniem
WF-11	Przeszukiwanie i sortowanie wpisów oraz obsługa z poziomu klawiatury
WF-12	Konfiguracja oraz obsługa fałszywego skarbca otwieranego pod przymusem

Źródło: opracowanie własne

Należy podkreślić, że zakres funkcjonalny został świadomie ograniczony zgodnie z przyjętą zasadą prymatu poprawności kryptograficznej nad liczbą funkcji. Każda funkcja, która rozszerzałaby powierzchnię ataku bez wyraźnej korzyści dla bezpieczeństwa lub użyteczności, została pominięta.

2.6 Pozycjonowanie własnego rozwiązania

Z przeglądu przedstawionego w podrozdziale 2.1 wynika, że poszczególne rozwiązania zajmują różne punkty na osi napięcia między wygodą synchronizacji chmurowej a minimalizacją modelu zaufania. Rozwiązanie opisane w niniejszej pracy świadomie sytuuje się po stronie minimalnego zaufania: jest menedżerem lokalnym zbudowanym zgodnie z zasadą zerowej wiedzy, w którym nawet obecny komponent serwerowy pozbawiony jest wiedzy o treści skarbca.

Tym, co odróżnia projekt od rozwiązań komercyjnych, jest połączenie trzech własności. Po pierwsze, **pełna przejrzystość**: kryptografia jest możliwa do prześledzenia na poziomie

kodu źródłowego, a jej poprawność weryfikowalna testami znanych odpowiedzi. Po drugie, **zaawansowane mechanizmy ochrony** wykraczające poza standard, z fałszywym skarbcem otwieranym pod przymusem na czele, cecha nieobecna w żadnym z przeglądanych rozwiązań. Po trzecie, **uczciwość w nazywaniu ograniczeń**, które potraktowano jako pełnoprawny przedmiot analizy, a nie jako element do przemilczenia. To właśnie te trzy własności wyznaczają wkład pracy i są przedmiotem szczegółowego omówienia w kolejnych rozdziałach.

Rozdział 3

PODSTAWY KRYPTOGRAFICZNE

Rozdział ten przedstawia teoretyczne i praktyczne podstawy warstwy kryptograficznej projektowanego systemu oraz uzasadnia dobór konkretnych prymitywów. Kolejno omówiono funkcje wyprowadzania klucza, szyfrowanie uwierzytelnione, opakowanie kluczy oraz spinający je model KEK/DEK. Każda decyzja została skonfrontowana z odrzuconymi alternatywami, a rozdział zamyka zestawienie kategorycznych zakazów wynikających z kontraktu kryptograficznego projektu. Przyjęto przy tym żelazną zasadę, że w systemie nie projektuje się własnych algorytmów kryptograficznych, wykorzystuje się wyłącznie sprawdzone, ustandaryzowane prymitywy o publicznie znanej analizie bezpieczeństwa [9, 13].

3.1 Wymagania wobec warstwy kryptograficznej

Warstwa kryptograficzna musi pogodzić kilka wymagań wynikających z modelu zerowej wiedzy. Po pierwsze, klucz chroniący skarbiec musi być wyprowadzany z hasła głównego w sposób na tyle kosztowny, by atak siłowy w trybie offline pozostawał nieopłacalny nawet po wycieku szyfrotekstu. Po drugie, samo szyfrowanie skarbca musi zapewniać nie tylko poufność, lecz także integralność, tak aby modyfikacja danych przez napastnika kontrolującego serwer była wykrywalna. Po trzecie, zmiana hasła głównego nie powinna pociągać za sobą ponownego szyfrowania całego skarbca. Po czwarte wreszcie, wszystkie prymitywy muszą być dostępne w środowisku przeglądarki, bezpośrednio poprzez interfejs WebCrypto [18] lub poprzez zwięzły moduł WebAssembly. Te cztery wymagania doprowadziły do wyboru, kolejno, funkcji Argon2id, trybu AES-256-GCM, algorytmu opakowania kluczy AES-KW oraz dwupoziomowego modelu KEK/DEK.

3.2 Funkcje wyprowadzania klucza

Funkcja wyprowadzania klucza (KDF) przekształca hasło o niskiej entropii w klucz kryptograficzny, czyniąc to celowo kosztownie, by spowolnić zgadywanie. Kluczowym kryterium różnicującym współczesne funkcje KDF jest *twardość pamięciowa* (ang. *memory-hardness*), wymóg zaangażowania znacznej ilości pamięci operacyjnej przy każdej próbie. Własność ta ma fundamentalne znaczenie obronne: o ile koszt obliczeniowy można tanio zrównoleglić na procesorach graficznych i układach specjalizowanych, o tyle koszt pamięciowy skaluje się znacznie gorzej, co istotnie podnosi cenę masowego ataku siłowego

[2, 22].

3.2.1 Argon2id jako funkcja główna

Jako podstawową funkcję wyprowadzania klucza wybrano **Argon2id**, zwycięzcę konkursu Password Hashing Competition [2], opisanego następnie w dokumencie RFC 9106 [3]. Argon2id łączy odporność na ataki czasowo-pamięciowe (wariant Argon2i) z odpornością na ataki wykorzystujące zależności danych (wariant Argon2d), stanowiąc rekomendowany domyślny tryb tej rodziny [3]. Ponieważ interfejs WebCrypto nie udostępnia Argon2id, funkcję tę dostarcza zwięzły moduł WebAssembly. Świadomą konsekwencją tego wyboru jest zależność od jednej dodatkowej biblioteki oraz konieczność dopuszczenia kompilacji modułu WebAssembly w polityce bezpieczeństwa treści, co omówiono w rozdziale 6.

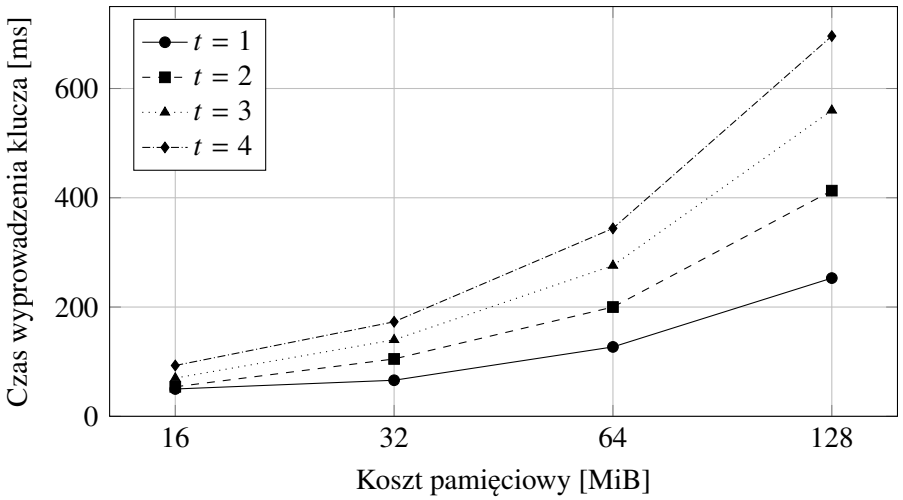
Parametry Argon2id, koszt pamięciowy m, liczba iteracji t oraz stopień równoległości p, nie są ustalone na sztywno, lecz **kalibrowane na urządzeniu** w chwili tworzenia skarbca. Procedura kalibracji dobiera parametry tak, by czas wyprowadzenia klucza mieścił się w przedziale około 0,5–1 sekundy, zwiększając najpierw koszt pamięciowy. Dobrane wartości zapisywane są w metadanych pakietu danych, dzięki czemu odblokowanie odtwarza dokładnie te same parametry. Tabela 3.1 prezentuje wyniki pomiaru czasu wyprowadzenia klucza na urządzeniu referencyjnym.

Tab. 3.1: Czas wyprowadzenia klucza funkcją Argon2id na urządzeniu referencyjnym ($p = 1$)

Koszt pamięciowy	$t = 1$	$t = 2$	$t = 3$	$t = 4$
16 MiB	50 ms	54 ms	70 ms	93 ms
32 MiB	66 ms	105 ms	140 ms	173 ms
64 MiB	127 ms	200 ms	276 ms	344 ms
128 MiB	253 ms	413 ms	560 ms	696 ms

Źródło: pomiar własny (`npm run bench`)

Na urządzeniu referencyjnym kalibracja wskazała jako zalecane parametry $m = 128\,\text{MiB}$, $t = 3$, $p = 1$, dające czas rzędu 560 ms. Wartości te są silnie zależne od sprzętu: na innym urządzeniu procedura dobierze inne, utrzymując zbliżony koszt czasowy. Konsekwencją tego podejścia jest fakt, że rzeczywista siła ochrony różni się między maszynami. Ograniczenie to omówiono w rozdziale 8. Te same dane przedstawiono graficznie na rysunku 3.1, który uwidacznia zbliżony do liniowego wzrost kosztu czasowego wraz z kosztem pamięciowym.



Rys. 3.1: Czas wyprowadzenia klucza Argon2id w funkcji kosztu pamięciowego oraz liczby iteracji t (urządzenie referencyjne, $p = 1$)
Źródło: pomiar własny (`npm run bench`)

3.2.2 PBKDF2 jako udokumentowany wariant zapasowy

Środowisko wykonawcze nie zawsze udostępnia WebAssembly. Może to wynikać ze starszej przeglądarki lub z restrykcyjnej polityki bezpieczeństwa treści. Aby aplikacja pozostawała funkcjonalna w takich warunkach, przewidziano udokumentowany wariant zapasowy: **PBKDF2-HMAC-SHA-256** [17] z liczbą 600 000 iteracji, zgodnie z zaleceniem organizacji OWASP [22]. Funkcja ta jest dostępna natywnie w interfejsie WebCrypto, nie wymaga więc żadnego dodatkowego modułu. Informacja o tym, która funkcja KDF została faktycznie użyta, zapisywana jest jawnie w metadanych w polu `nazwa`, dzięki czemu odblokowanie wybiera właściwą funkcję na podstawie zawartości pakietu.

Należy uczciwie zaznaczyć, że PBKDF2 jest istotnie słabszy od Argon2id, ponieważ **nie jest funkcją twardą pamięciowo**: jego koszt sprowadza się do liczby iteracji i daje się tanio zrównoleglić sprzętowo. Wariant zapasowy stanowi zatem świadomy kompromis pomiędzy zasadą „działa wszędzie” a zasadą „maksymalny koszt ataku”. Ponowna kalibracja na urządzeniu wyposażonym w WebAssembly przełącza skarbiec z powrotem na Argon2id. Odrzucono natomiast scrypt, choć również twardy pamięciowo [22], jest on słabiej wspierany w środowisku przeglądarki niż Argon2id dostarczany przez moduł WebAssembly. Listing 3.1 przedstawia strukturę metadanych KDF zapisywanych w pakiecie danych.

```typescript
// Sól jest jawna (nie jest sekretem); pole ‘nazwa‘ rozstrzyga, któ
    rej
// funkcji KDF użyto, aby odblokowanie odtworzyło właściwy wariant.
interface MetadaneArgon2id {
  readonly nazwa: "argon2id";
  readonly sol: string;   // base64
  readonly m: number;     // koszt pamięciowy (KiB)
  readonly t: number;     // liczba iteracji
  readonly p: number;     // równoległość
}

interface MetadanePbkdf2 {
  readonly nazwa: "pbkdf2";
  readonly sol: string;     // base64
  readonly iteracje: number; // np. 600 000
}

type MetadaneKdf = MetadaneArgon2id | MetadanePbkdf2;
```

Listing 3.1: Metadane KDF zapisywane w pakiecie danych (TypeScript)
Źródło: opracowanie własne

3.3 Szyfrowanie uwierzytelnione AES-256-GCM

Właściwą treść skarbca szyfruje algorytm **AES-256-GCM** [6] udostępniany przez interfejs WebCrypto [18]. Tryb Galois/Counter Mode należy do klasy szyfrowania uwierzytelnionego (AEAD): obok poufności zapewnia integralność, generując *znacznik uwierzytelniający*, którego weryfikacja przy deszyfrowaniu wykrywa każdą modyfikację szyfrotekstu. Ma to bezpośrednie znaczenie w modelu zagrożeń: napastnik kontrolujący serwer, który zmieniłby choćby jeden bajt szyfrotekstu, spowoduje niepowodzenie weryfikacji znacznika, a klient odmówi przyjęcia danych zamiast operować na przekłamanej treści.

Bezwzględnie przestrzeganą zasadą jest stosowanie **świeżego, losowego 96-bitowego wektora inicjującego** dla każdej operacji szyfrowania. Długość 96 bitów jest zalecaną długością wektora dla trybu GCM [6], natomiast wymóg świeżości wynika z natury tego trybu: powtórne użycie tej samej pary klucz–wektor katastrofalnie osłabia bezpieczeństwo, umożliwiając odzyskanie informacji o tekście jawnym oraz podważenie integralności [6, 9]. Wektor inicjujący generowany jest kryptograficznie bezpiecznym generatorem liczb losowych i zapisywany jawnie w metadanych, nie jest on sekretem. Listing 3.2 pokazuje rdzeń operacji szyfrowania.

```
const DLUGOSC_IV = 12; // 96 bitów, zalecane dla AES-GCM

async function zaszyfrujDane(dek: CryptoKey, dane: unknown) {
  // Świeży, losowy wektor inicjujący dla KAŻDEGO szyfrowania.
  const iv = crypto.getRandomValues(new Uint8Array(DLUGOSC_IV));
  const jawny = tekstNaBajty(JSON.stringify(dane));
  const wynik = await crypto.subtle.encrypt({ name: "AES-GCM", iv },
      dek, jawny);
  return { iv, szyfrotekst: new Uint8Array(wynik) };
}
```

Listing 3.2: Szyfrowanie danych ze świeżym wektorem inicjującym (TypeScript)
Źródło: opracowanie własne

Każdy zapis skarbca generuje zatem nowy wektor inicjujący, co potwierdza dedykowany test sprawdzający świeżość wektora pomiędzy kolejnymi zapisami. Świadomie odrzucono tryby pozbawione uwierzytelnienia, takie jak CBC czy CTR, a także, co oczywiste, tryb ECB oraz jakiekolwiek użycie stałego wektora inicjującego [9, 13].

3.4 Opakowanie kluczy AES-KW

Do opakowania klucza danych zastosowano algorytm **AES-KW** opisany w dokumencie RFC 3394 [26]. Jest to dedykowany, uwierzytelniony prymityw przeznaczony wyłącznie do opakowywania materiału kluczowego. Jego zasadniczą zaletą w kontekście niniejszego projektu jest brak konieczności zarządzania wektorem inicjującym: AES-KW jest deterministyczny i posługuje się stałą wartością kontrolną (ICV) wbudowaną w algorytm, co całkowicie eliminuje klasyczną klasę błędów polegającą na powtórnym użyciu wektora. Jednocześnie zachowuje on własność uwierzytelnienia, próba odpakowania kluczem wyprowadzonym z błędnego hasła kończy się jawnym niepowodzeniem, co stanowi podstawę mechanizmu rozróżniania poprawnego i niepoprawnego hasła.

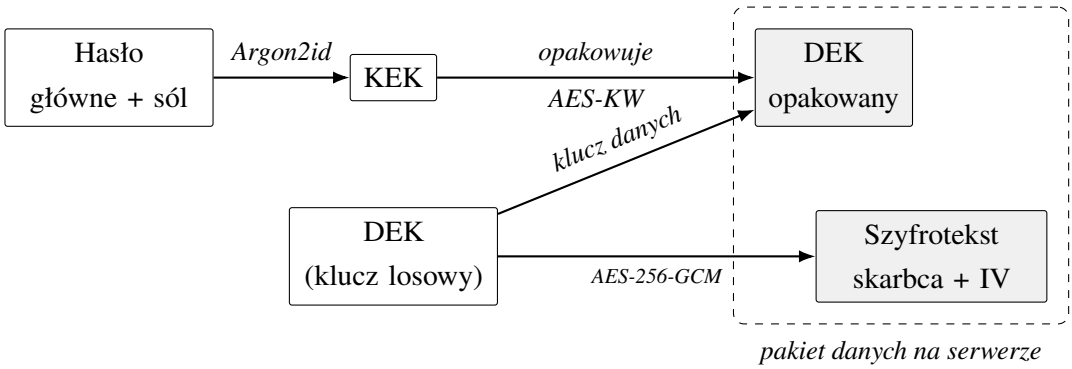
Odrzucono dwie alternatywy. Użycie pojedynczego klucza wyprowadzonego wprost z hasła, bez warstwy klucza danych, wymagałoby ponownego zaszyfrowania całego skarbca przy każdej zmianie hasła. Z kolei zastosowanie trybu AES-GCM do opakowania klucza danych ponownie wprowadziłoby konieczność zarządzania wektorem inicjującym, której AES-KW pozwala uniknąć.

3.5 Model KEK/DEK

Spoiwem opisanych prymitywów jest dwupoziomowy **model KEK/DEK**. Klucz wyprowadzony z hasła głównego, zwany kluczem opakowującym (KEK), nie szyfruje bezpośrednio danych, służy wyłącznie do opakowania losowego klucza danych (DEK), który dopiero szyfruje skarbiec algorytmem AES-256-GCM. Zależności pomiędzy elementami modelu można zapisać następująco:

- `KEK = KDF(hasło, sól)`, klucz opakowujący wyprowadzony z hasła,
- `DEK`, losowo wygenerowany klucz danych (AES-256-GCM),
- `DEK_opak. = AES-KW(DEK, KEK)`, opakowana postać klucza danych.

Na serwer trafia wyłącznie opakowana postać klucza danych, `DEK_opak.`, będąca dla serwera nieprzejrzystym ciągiem bajtów. Rysunek 3.2 ilustruje przepływ danych w tym modelu.



Rys. 3.2: Model KEK/DEK, klucz wyprowadzony z hasła (KEK) opakowuje losowy klucz danych (DEK), który szyfruje skarbiec
Źródło: opracowanie własne

Najważniejszą korzyścią praktyczną tej konstrukcji jest sposób obsługi zmiany hasła. Ponieważ to klucz danych, a nie klucz wyprowadzony z hasła, szyfruje skarbiec, zmiana hasła głównego sprowadza się do odpakowania klucza danych starym kluczem opakowującym i ponownego opakowania go nowym, wyprowadzonym ze świeżej soli. Sam szyfrotekst skarbca pozostaje przy tym nietknięty. Listing 3.3 przedstawia tę operację.

```
// Odpakuj DEK starym KEK (eksportowalny tylko po to, by go ponownie opakować),
// a następnie opakuj go nowym KEK wyprowadzonym ze świeżej soli.
const staryKek = await odtworzKek(stareHaslo, docelowy.kdf);
const dek = await odpakujDek(base64NaBajty(docelowy.dekOpakowany), staryKek, true);

const { kek: nowyKek, kdf } = await utworzKek(noweHaslo, nowePar);
const nowyOpakowany = await crypto.subtle.wrapKey("raw", dek, nowyKek, "AES-KW");
// Szyfrotekst skarbca pozostaje nietknięty, zmieniają się tylko metadane KDF
// oraz opakowana postać klucza danych.
```

Listing 3.3: Zmiana hasła jako ponowne opakowanie klucza danych (TypeScript)
Źródło: opracowanie własne

Model KEK/DEK ma jeszcze jedno istotne następstwo dotyczące eksportowalności kluczy. Klucz opakowujący oraz sesyjny klucz danych są nieeksportowalne, a klucz danych

przyjmuje postać eksportowalną wyłącznie na krótką chwilę tworzenia lub zmiany hasła, gdy musi zostać zserializowany na potrzeby operacji opakowania. Rozdzielenie ról kluczy umożliwia ponadto zaawansowane mechanizmy ochrony, takie jak rotacja klucza danych, którą omówiono w rozdziale 5.

3.6 Kontrakt kryptograficzny i zakazy

Powyższe wybory składają się na zablokowany *kontrakt kryptograficzny* projektu, którego nie wolno naruszyć. Tabela 3.2 zestawia jego elementy.

Tab. 3.2: Kontrakt kryptograficzny projektu

Element	**Wybór**
Funkcja KDF	Argon2id (RFC 9106), kalibrowana na urządzeniu; PBKDF2-HMAC-SHA-256 jako udokumentowany wariant zapasowy
Szyfrowanie skarbca	AES-256-GCM ze świeżym, losowym 96-bitowym wektorem inicjującym dla każdej operacji
Opakowanie klucza	AES-KW (RFC 3394), uwierzytelnione, bez wektora inicjującego
Model klucza	dwupoziomowy KEK/DEK; zmiana hasła = ponowne opakowanie klucza danych
Metadane	sól, parametry KDF, wektor inicjujący, wersja, przechowywane jawnie obok szyfrotekstu

Źródło: opracowanie własne na podstawie decyzji projektowych

Z kontraktu wynikają kategoryczne zakazy. Niedopuszczalne jest projektowanie własnej kryptografii lub własnej funkcji KDF, stosowanie trybu ECB, użycie szyfrowania bez uwierzytelnienia, wykorzystywanie funkcji MD5 lub SHA-1 w roli funkcji wyprowadzania klucza, a także posługiwanie się zakodowanymi na sztywno solami lub kluczami oraz stałym wektorem inicjującym. Zakazy te nie są arbitralne, każdy z nich odpowiada dobrze udokumentowanej klasie błędów kryptograficznych [9, 23, 27], a ich przestrzeganie jest warunkiem wiarygodności całego rozwiązania. Poprawność faktycznej implementacji opisanych prymitywów potwierdzają testy znanych odpowiedzi, omówione w rozdziale 7.

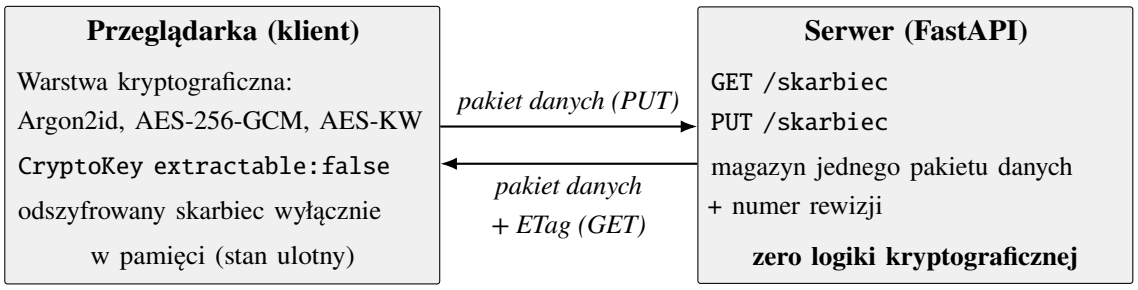
Rozdział 4

ARCHITEKTURA SYSTEMU

Rozdział ten przedstawia architekturę systemu jako całości, łączącą warstwę kryptograficzną z rozdziału 3 z konkretnym podziałem na komponent kliencki i serwerowy. Omówiono podział odpowiedzialności w modelu zerowej wiedzy, format przechowywanego pakietu danych wraz z jego dwuslotową strukturą, rolę serwera jako pasywnego magazynu danych, mechanizm optymistycznej kontroli współbieżności, nagłówki bezpieczeństwa oraz model dostępu. Rozdział zamykają opisy przepływów tworzenia i odblokowania skarbca, ilustrujące współdziałanie obu komponentów.

4.1 Przegląd architektury

System składa się z dwóch komponentów: aplikacji klienckiej działającej w przeglądarce oraz lekkiego serwera pełniącego rolę magazynu. Cała kryptografia, wyprowadzenie klucza, szyfrowanie, deszyfrowanie oraz zaawansowane mechanizmy ochrony, żyje wyłącznie po stronie klienta. Serwer wymienia z klientem jedynie nieprzejrzysty pakiet danych i nie wykonuje żadnej operacji kryptograficznej. Rysunek 4.1 przedstawia ogólny podział systemu.



Rys. 4.1: Ogólny podział systemu w modelu zerowej wiedzy
Źródło: opracowanie własne

Zasadą organizującą całość jest stwierdzenie, że **serwer jest pasywnym magazynem bajtów**: nie szyfruje, nie deszyfruje i nie zna hasła. Taki podział sprawia, że powierzchnia zaufania serwera jest minimalna, a jego ewentualna kompromitacja nie narusza poufności danych.

4.2 Podział odpowiedzialności klient–serwer

Tabela 4.1 zestawia odpowiedzialności obu komponentów. Podział ten jest bezpośrednią konsekwencją żelaznych zasad projektu: wszystko, co dotyczy sekretów, należy do klienta, a serwerowi pozostaje wyłącznie trwałe przechowywanie i udostępnianie nieprzejrzystych bajtów.

Tab. 4.1: Podział odpowiedzialności pomiędzy klienta a serwer

Klient (przeglądarka)	**Serwer (FastAPI)**
Wyprowadzenie klucza z hasła (KDF)	Przechowanie nieprzejrzystego pakietu danych
Szyfrowanie i deszyfrowanie skarbca	Zwracanie pakietu wraz ze znacznikiem rewizji
Zarządzanie kluczami (KEK/DEK)	Egzekwowanie warunkowego zapisu (współbieżność)
Zaawansowane mechanizmy ochrony	Walidacja kształtu pakietu (`extra=forbid`)
Renderowanie interfejsu użytkownika	Ustawianie nagłówków bezpieczeństwa

Źródło: opracowanie własne

Co istotne, żadna pozycja z kolumny serwera nie wymaga znajomości treści skarbca. Walidacja kształtu pakietu czy kontrola współbieżności operują wyłącznie na metadanych i nieprzejrzystych ciągach znaków, nigdy na danych jawnych.

4.3 Format pakietu danych

Sercem architektury jest *format pakietu danych* (ang. *envelope*), samowystarczalna struktura, którą serwer przechowuje jako nieprzejrzysty obiekt. Pakiet zawiera wszystko, co jest potrzebne do odtworzenia klucza i odszyfrowania skarbca, prócz samego hasła. Tabela 4.2 opisuje jego pola.

Tab. 4.2: Pola pakietu danych

Pole	Znaczenie
`wersja`	Numer wersji formatu, umożliwiający przyszłą migrację metadanych
`kdf`	Nieprzejrzyste metadane funkcji KDF: nazwa, sól, parametry
`dekOpakowany`	Klucz danych opakowany kluczem opakowującym (base64)
`iv`	Wektor inicjujący dla szyfrowania skarbca (base64)
`szyfrotekst`	Zaszyfrowana treść skarbca (base64)
`przyneta`	Opcjonalny drugi slot, fałszywy skarbiec „pod przymusem”

Źródło: opracowanie własne na podstawie formatu danych projektu

Cechą szczególną formatu jest jego **dwuslotowa struktura**. Slot główny leży „na płasko” w pakiecie, co zapewnia zgodność wsteczną z formatem jednoskarbcowym, natomiast opcjonalne pole `przyneta` stanowi drugi, niezależnie zaszyfrowany slot otwierany innym hasłem. Każdy slot dysponuje własnym kompletem metadanych KDF, opakowanym kluczem danych, wektorem inicjującym oraz szyfrotekstem, dzięki czemu jest szyfrowany w pełni niezależnie, a otwarcie jednego nie ujawnia niczego o drugim. Mechanizm ten jest podstawą fałszywego skarbca, omówionego w rozdziale 5. Listing 4.1 przedstawia typ pojedynczego slotu.

```
interface SlotSkarbca {
  readonly kdf: MetadaneKdf;     // nieprzejrzyste metadane KDF slotu
  readonly dekOpakowany: string; // DEK opakowany KEK-iem (AES-KW), base64
  readonly iv: string;           // wektor inicjujący AES-GCM, base64
  readonly szyfrotekst: string;  // zaszyfrowane dane slotu, base64
}

interface ZaszyfrowanySkarbiec extends SlotSkarbca {
  readonly wersja: 1;
  readonly przyneta?: SlotSkarbca; // opcjonalny fałszywy skarbiec
}
```

Listing 4.1: Typ pojedynczego slotu pakietu danych (TypeScript)
Źródło: opracowanie własne

4.4 Serwer jako pasywny magazyn danych

Komponent serwerowy zrealizowano w technologii FastAPI [8] i ograniczono do dwóch zasadniczych operacji: pobrania oraz zapisania pakietu. Punkt końcowy `GET /skarbiec` zwraca przechowywany pakiet wraz ze znacznikiem rewizji, natomiast `PUT /skarbiec` zastępuje pakiet w całości. Ponieważ rozwiązanie obsługuje pojedynczego użytkownika, magazyn przechowuje dokładnie jeden skarbiec utrwalany w bazie SQLite jako tekst. Serwer nigdy go nie deszyfruje.

Kluczowym elementem obrony w głąb jest **twarda walidacja kształtu pakietu**. Schemat danych skonfigurowano z opcją `extra="forbid"`, co oznacza, że serwer odrzuca każde żądanie zawierające pole spoza zdefiniowanego formatu. Gdyby na skutek błędu klienta do pakietu trafiło pole takie jak `haslo`, żądanie zostałoby odrzucone, a hasło nigdy nie trafiłoby do magazynu. Mechanizm ten jest praktycznym zabezpieczeniem żelaznej zasady mówiącej, że hasło nie opuszcza przeglądarki. Listing 4.2 pokazuje definicję schematu serwera.

```python
class Skarbiec(BaseModel):
    # extra="forbid", serwer ODMAWIA zapisania czegokolwiek poza
        envelope'em;
    # próba dosłania np. pola ‘haslo‘ zostanie odrzucona (422).
    model_config = ConfigDict(extra="forbid")

    wersja: int = Field(ge=1)
    kdf: dict[str, Any]          # nieprzejrzyste metadane KDF
    dekOpakowany: str = Field(min_length=1)
    iv: str = Field(min_length=1)
    szyfrotekst: str = Field(min_length=1)
    przyneta: Slot | None = Field(default=None)
```

Listing 4.2: Schemat pakietu z odrzucaniem obcych pól (Python, Pydantic)
Źródło: opracowanie własne

Należy podkreślić, że pole `kdf` jest dla serwera nieprzejrzystym obiektem, nie wie on i nie musi wiedzieć, w jaki sposób klient wyprowadził klucz. To rozgraniczenie utrzymuje serwer w roli magazynu nawet wobec przyszłych zmian w warstwie kryptograficznej klienta.

4.5 Kontrola współbieżności

Choć rozwiązanie obsługuje jednego użytkownika, może on korzystać z aplikacji w wielu kartach lub na wielu urządzeniach. Powstaje wówczas ryzyko *cichego nadpisania* zmian: dwie sesje pobierają ten sam pakiet, a następnie zapisują go kolejno, przy czym druga sesja nieświadomie usuwa zmiany wprowadzone przez pierwszą. Aby temu zapobiec, zastosowano **optymistyczną kontrolę współbieżności** opartą na mechanizmie znaczników

rewizji.

Każdy zapis zwiększa licznik rewizji przechowywany obok pakietu. Operacja `GET` zwraca bieżącą rewizję w nagłówku `ETag`, a klient dołącza ją do kolejnego zapisu w nagłówku `If-Match`; oba nagłówki oraz semantykę żądań warunkowych definiuje standard HTTP [10]. Jeżeli rewizja podana przez klienta nie odpowiada aktualnej, co oznacza, że w międzyczasie ktoś inny zapisał pakiet, serwer odrzuca żądanie kodem `412 Precondition Failed`, zwracając aktualną rewizję, zamiast nadpisać cudze zmiany. Należy zaznaczyć, że licznik rewizji nie jest sekretem ani elementem kryptografii; to zwykły licznik zapisów, dzięki czemu serwer pozostaje pasywnym magazynem danych. Listing 4.3 przedstawia rdzeń tej logiki.

```
def zapisz(envelope, oczekiwana_rewizja):
    obecna = ... # bieżąca rewizja z bazy
    # Zapis warunkowy: przy nieaktualnej rewizji zgłoś konflikt
       zamiast nadpisać.
    if oczekiwana_rewizja is not None and obecna !=
       oczekiwana_rewizja:
        raise KonfliktWersji(obecna)  # serwer odpowie 412 z
            aktualnym ETag
    nowa = obecna + 1
    ...  # utrwal envelope z nową rewizją
    return nowa
```

Listing 4.3: Optymistyczna kontrola współbieżności (Python)
Źródło: opracowanie własne

4.6 Nagłówki bezpieczeństwa i model dostępu

Mimo że serwer nie wykonuje kryptografii, to właśnie on odpowiada za nagłówki bezpieczeństwa towarzyszące serwowanej aplikacji [19, 23]. W architekturze zerowej wiedzy front jest powierzchnią ataku numer jeden, dlatego nagłówki te mają znaczenie pierwszorzędne. Tabela 4.3 zestawia najważniejsze z nich.

Tab. 4.3: Wybrane nagłówki bezpieczeństwa ustawiane przez serwer

Nagłówek	Rola
`Content-Security-Policy`	Ścisła polityka treści (`default-src 'none'`) ograniczająca źródła skryptów i zasobów
`X-Content-Type-Options`	Wyłączenie zgadywania typu treści (`nosniff`)
`Referrer-Policy`	Brak wycieku adresów przez nagłówek odsyłacza (`no-referrer`)
`Cache-Control`	`no-store` dla danych skarbca, zakaz buforowania
`Permissions-Policy`	Wyłączenie nieużywanych, wrażliwych funkcji przeglądarki

Źródło: opracowanie własne na podstawie konfiguracji serwera

Polityka bezpieczeństwa treści jest na tyle istotna, że poświęcono jej osobne omówienie w rozdziale 6. W zakresie kontroli dostępu serwer domyślnie nasłuchuje wyłącznie na interfejsie pętli zwrotnej (`127.0.0.1`), co czyni go niedostępnym z sieci zewnętrznej. Gdy zachodzi potrzeba wystawienia go na innym interfejsie, możliwe jest włączenie opcjonalnego tokenu dostępu przekazywanego zmienną środowiskową: każde żądanie musi wówczas podać go w nagłówku `Authorization`, a porównanie odbywa się w czasie stałym. Token chroni jednak wyłącznie integralność i dostępność zapisu, nie zaś poufność, ta jest zapewniona przez samo szyfrowanie pakietu, niezależnie od kontroli dostępu.

4.7 Przepływy tworzenia i odblokowania skarbca

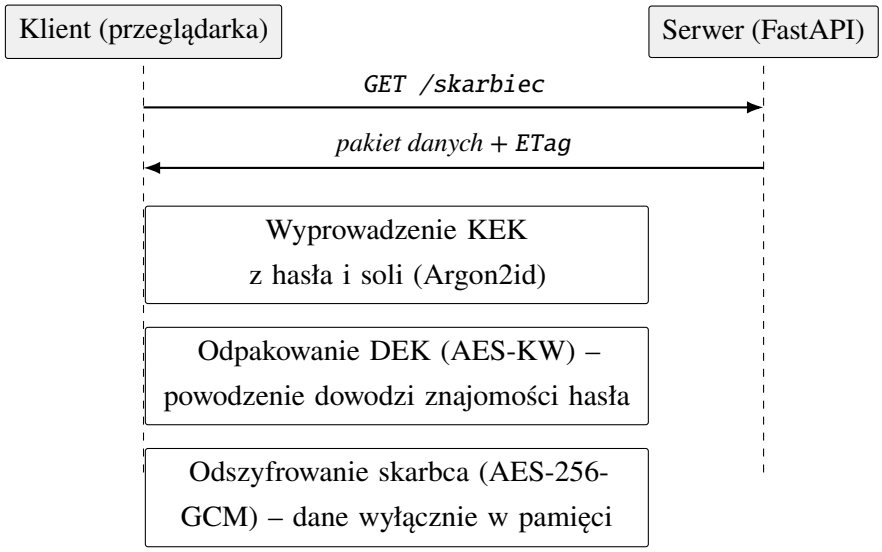
Współdziałanie komponentów najlepiej ilustrują dwa podstawowe przepływy. Pierwszym jest **utworzenie skarbca**, realizowane w całości po stronie klienta przed wysłaniem czegokolwiek do serwera:

1. użytkownik podaje hasło główne, którego siłę weryfikuje bramka entropii;
2. klient generuje losową sól oraz kalibruje parametry funkcji Argon2id;
3. z hasła i soli wyprowadzany jest klucz opakowujący (KEK);
4. generowany jest losowy klucz danych (DEK) i opakowywany kluczem KEK;
5. dane początkowe są szyfrowane kluczem danych ze świeżym wektorem inicjującym;
6. kompletny pakiet trafia do serwera operacją `PUT`, hasło pozostaje wyłącznie w przeglądarce.

Drugim przepływem jest **odblokowanie skarbca**, w którym „logowanie” sprowadza się do lokalnego odszyfrowania:

1. klient pobiera pakiet operacją `GET`, odczytując zarazem rewizję;
2. z podanego hasła oraz zapisanej soli i parametrów odtwarzany jest klucz opakowujący;
3. podejmowana jest próba odpakowania klucza danych, jej powodzenie jest dowodem znajomości prawidłowego hasła;
4. klucz danych deszyfruje skarbiec; niezgodność znacznika uwierzytelniającego oznacza naruszenie integralności i skutkuje odmową;
5. odszyfrowany skarbiec istnieje odtąd wyłącznie w pamięci aplikacji.

Przepływ odblokowania, z podziałem na jedyną wymianę z serwerem oraz operacje wykonywane lokalnie, przedstawia rysunek 4.2.



Rys. 4.2: Przepływ odblokowania skarbca, po jednym pobraniu pakietu cała reszta odbywa się po stronie klienta; hasło główne nigdy nie trafia do serwera
Źródło: opracowanie własne

W żadnym z przepływów hasło główne ani klucze nie są przesyłane do serwera. Gdy pakiet zawiera dodatkowo slot przynęty, krok odtworzenia klucza wykonywany jest dla obu slotów równolegle, tak aby czas odblokowania nie zdradzał, które hasło podano, szczegóły tego mechanizmu omówiono w rozdziale 5.

Rozdział 5

ZAAWANSOWANE MECHANIZMY OCHRONY

Rozdział ten stanowi rdzeń pracy. Omówiono w nim tytułowe *zaawansowane mechanizmy ochrony*, czyli zbiór rozwiązań wykraczających poza standardowy zakres menedżera haseł, które razem podnoszą odporność systemu na zagrożenia zidentyfikowane w rozdziale 2. Każdy mechanizm przedstawiono według jednolitego schematu: najpierw nazwano **zagrożenie**, na które odpowiada, następnie opisano **projekt rozwiązania**, po czym przeprowadzono jego **analizę** oraz, co stanowi istotny element rzetelności pracy, wskazano **uczciwe ograniczenia**. Centralne miejsce zajmuje fałszywy skarbiec otwierany pod przymusem, najbardziej wyróżniający mechanizm całego rozwiązania. Działanie opisanych dalej mechanizmów w gotowej aplikacji dokumentują zrzuty ekranu zebrane w dodatku D.

5.1 Fałszywy skarbiec „pod przymusem”

Zagrożenie: Klasyczna kryptografia chroni dane przed napastnikiem, który nie zna hasła. Nie odpowiada jednak na scenariusz *wymuszenia*, w którym napastnik zmusza użytkownika do otwarcia skarbca, żądając „pokaż, co masz”. W takiej sytuacji najsilniejszy nawet algorytm jest bezużyteczny, ponieważ to sam uprawniony użytkownik dokonuje odszyfrowania pod naciskiem.

Projekt rozwiązania: Odpowiedzią jest **fałszywy skarbiec**, wiarygodny, lecz niegroźny skarbiec, który użytkownik może okazać napastnikowi zamiast prawdziwego. Mechanizm zrealizowano jako opcjonalny **drugi slot** pakietu danych, opisany w rozdziale 4. Slot przynęty dysponuje własnym kompletem metadanych KDF, własnym kluczem danych, wektorem inicjującym oraz szyfrotekstem, jest więc szyfrowany w pełni niezależnie od slotu głównego i otwierany innym hasłem. Wskutek tej niezależności otwarcie jednego slotu nie ujawnia żadnej informacji o istnieniu ani zawartości drugiego.

Kluczowym elementem konstrukcji jest **jednakowy czas odblokowania** niezależny od tego, które hasło podano. Gdyby klient najpierw próbował otworzyć slot główny, a dopiero po niepowodzeniu slot przynęty, czas reakcji zdradzałby, które hasło zostało wprowadzone, niwecząc cały mechanizm. Aby temu zapobiec, klucze opakowujące obu slotów wyprowadzane są **zawsze i równolegle**, niezależnie od tego, który slot ostatecznie zostanie otwarty. Ponieważ wyprowadzenie klucza funkcją Argon2id jest operacją dominującą czasowo, jej wykonanie dla obu slotów wyrównuje czas odblokowania. Listing 5.1 przedstawia tę logikę.

```
// Wyprowadź WSZYSTKIE KEK równolegle, koszt jest jednakowy niezależnie od tego,
// który slot zostanie trafiony, więc czas odblokowania nie zdradza hasła.
const keki = await Promise.all(sloty.map(({ slot }) => odtworzKek(haslo, slot.kdf)));

for (let i = 0; i < sloty.length; i += 1) {
  const wynik = await otworzSlot(sloty[i].slot, keki[i]);
  if (wynik !== null) {
    return { dek: wynik.dek, dane: wynik.dane, slot: sloty[i].id };
  }
}
throw new BledneHaslo();
```

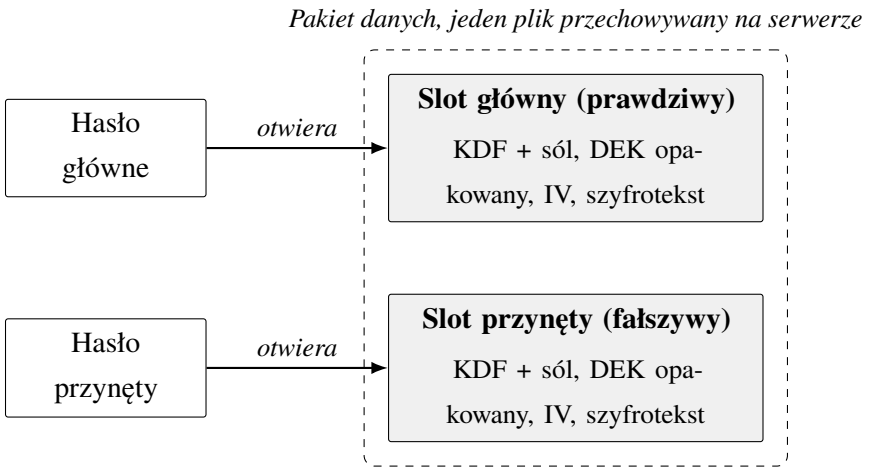
Listing 5.1: Równoległe wyprowadzenie kluczy obu slotów dla jednakowego czasu odblokowania (TypeScript)
Źródło: opracowanie własne

Konfiguracja przynęty jest możliwa wyłącznie z poziomu prawdziwej sesji, a hasło przynęty musi różnić się od hasła głównego, w przeciwnym razie nie dałoby się rozróżnić slotów. Strażnik weryfikuje ten warunek przy tworzeniu przynęty, sprawdzając, że jej hasło nie otwiera slotu głównego (listing 5.2).

```
// Hasło fałszywego skarbca nie może otwierać slotu głównego, inaczej sloty
// byłyby nierozróżnialne.
const kekGlowny = await odtworzKek(hasloPrzynety, paczka.kdf);
const trafiaWGlowny = await otworzSlot(paczka, kekGlowny)
  .then((w) => w !== null, () => false);
if (trafiaWGlowny) {
  throw new Error("Hasło fałszywego skarbca musi różnić się od hasła głównego.");
}
```

Listing 5.2: Strażnik różności haseł przy tworzeniu przynęty (TypeScript)
Źródło: opracowanie własne

Dwuslotową strukturę pakietu danych oraz odwzorowanie haseł na sloty ilustruje rysunek 5.1.



Rys. 5.1: Dwuslotowa konstrukcja fałszywego skarbca, oba sloty szyfrowane niezależnie, otwierane różnymi hasłami; KEK obu slotów wyprowadzane są zawsze, co zapewnia jednakowy czas odblokowania

Źródło: opracowanie własne

Analiza: Mechanizm zapewnia praktyczną, wiarygodną ochronę przed wymuszeniem: pod naciskiem użytkownik podaje hasło przynęty, otwierając niegroźny skarbiec, który wygląda jak prawdziwy. Niezależne szyfrowanie obu slotów sprawia, że napastnik obserwujący jedynie zachowanie aplikacji nie jest w stanie stwierdzić, czy otwarto skarbiec prawdziwy, czy fałszywy, ani czy w ogóle istnieje drugi slot.

Uczciwe ograniczenie: Należy jednak wyraźnie zaznaczyć granicę tego rozwiązania. Fałszywy skarbiec jest ochroną przed **wymuszeniem**, a **nie** kryptograficzną *zaprzeczalnością* (ang. *deniability*). Napastnik, który zbada nie zachowanie aplikacji, lecz surowy pakiet danych, i zna jego format, dostrzeże obecność pola `przyneta`, a więc samego faktu istnienia drugiego slotu. Prawdziwa nieodróżnialność, znana z rozwiązań typu ukryty wolumin (jak w programie VeraCrypt), wymaga znacznie bardziej złożonej konstrukcji, w której obecność ukrytych danych jest matematycznie niewykrywalna. Konstrukcja taka jest istotnie trudniejsza i ryzykowna w implementacji, dlatego świadomie nie została zrealizowana. Odrzucono również rozwiązanie pośrednie, zawsze obecny slot wypełniony losowym szumem, jako dające fałszywe poczucie zaprzeczalności przy wciąż obecnej wykrywalności forensycznej.

5.2 Rotacja klucza danych

Zagrożenie: Możliwy jest scenariusz, w którym istnieje podejrzenie, że klucz danych mógł wyciec z pamięci, na przykład wskutek chwilowej kompromitacji środowiska wykonawczego. Sama zmiana hasła nie rozwiązuje tego problemu, ponieważ, jak opisano w rozdziale 3, zmienia ona jedynie opakowanie tego samego klucza danych, który w dalszym ciągu szyfruje skarbiec.

Projekt rozwiązania: Mechanizm **rotacji klucza danych** generuje całkowicie nowy klucz danych i ponownie szyfruje nim cały skarbiec, używając przy tym świeżej soli oraz świeżego wektora inicjującego. W odróżnieniu od zmiany hasła, po rotacji stary klucz danych przestaje cokolwiek odszyfrowywać. Hasło główne pozostaje niezmienione, jest jednak weryfikowane przed rotacją, aby operacja nie pozostawiła skarbca w stanie niedostępnym. Drugi slot, jeśli istnieje, pozostaje nietknięty.

Analiza i ograniczenie: Rotacja stanowi mechanizm obrony w głąb stosowany po podejrzeniu wycieku klucza: unieważnia ona wszelkie kopie starego klucza danych. Należy zaznaczyć, że jej skuteczność opiera się na założeniu, iż samo środowisko nie jest w chwili rotacji nadal skompromitowane, w przeciwnym razie napastnik mógłby pozyskać również nowy klucz. Mechanizm chroni zatem przed wykorzystaniem wcześniej wykradzionego materiału kluczowego, nie zaś przed trwającą aktywnie kompromitacją.

5.3 Bramka siły hasła głównego

Zagrożenie: Hasło główne jest jedynym punktem awarii całego systemu. Najsilniejsza funkcja wyprowadzania klucza nie uratuje skarbca chronionego hasłem o niskiej entropii, ponieważ atakujący zgadnie takie hasło, zanim koszt funkcji KDF stanie się istotny.

Projekt rozwiązania: Bramka siły hasła głównego szacuje entropię hasła w chwili jego tworzenia lub zmiany i uzależnia od niej możliwość kontynuowania. Entropię oszacowano w modelu przestrzeni poszukiwań [11]: liczba bitów jest iloczynem długości hasła i logarytmu dwójkowego rozmiaru puli znaków, przy czym pula rośnie wraz z liczbą użytych klas znaków (małe litery, wielkie litery, cyfry, symbole). Listing 5.3 przedstawia rdzeń oszacowania, a tabela 5.1, progi decyzyjne.

```
1 // Model przestrzeni poszukiwań: bity = długość * log2(rozmiar puli
      znaków).
2 let rozmiarPuli = 0;
3 for (const klasa of KLASY_ZNAKOW) {     // 26 + 26 + 10 + 33
4   if (klasa.wzor.test(haslo)) {
5     rozmiarPuli += klasa.rozmiar;
6   }
7 }
8 return Math.round(haslo.length * Math.log2(rozmiarPuli));
```

Listing 5.3: Szacowanie entropii hasła w bitach (TypeScript)
Źródło: opracowanie własne

Tab. 5.1: Progi siły hasła głównego i reakcja bramki

Entropia	Ocena	Reakcja bramki
poniżej 40 bitów	słabe	zablokowanie utworzenia
40–60 bitów	umiarkowane	wymóg świadomego potwierdzenia
60–80 bitów	dobre	dopuszczone
80 bitów i więcej	silne	dopuszczone

Źródło: opracowanie własne na podstawie progów implementacji

Analiza: Bramka wprowadza lekkie tarcie wyłącznie tam, gdzie jest ono uzasadnione, przy słabym haśle i tylko podczas tworzenia lub zmiany. Hasła wyraźnie zbyt słabe są blokowane, hasła umiarkowane wymagają świadomego potwierdzenia, a hasła dobre i silne przechodzą bez przeszkód. Progi są spójne z miernikiem entropii prezentowanym w interfejsie, który jest jednocześnie elementem-sygnaturą systemu projektowego.

Uczciwe ograniczenie: Zastosowany model entropii daje **górne oszacowanie** siły hasła. Zakłada on, że znaki są dobierane losowo w wykrytej puli, nie wykrywa natomiast słów słownikowych, sekwencji klawiaturowych ani powtórzeń. Hasło takie jak `Password1!` otrzyma w tym modelu wysoki wynik mimo swojej faktycznej słabości, na co zwracają uwagę wytyczne dotyczące oceny siły haseł [11, 22]. Miernik ma zatem charakter zachęty do haseł dłuższych i bardziej zróżnicowanych, nie zaś gwarancji bezpieczeństwa. Świadomie zrezygnowano z bardziej rozbudowanej analizy słownikowej, która zwiększyłaby liczbę zależności i powierzchnię ataku łańcucha dostaw.

5.4 Auto-blokada po bezczynności

Zagrożenie: Odblokowany skarbiec istnieje w pamięci wraz z kluczem danych. Pozostawienie otwartej sesji bez nadzoru, na przykład przy odejściu od urządzenia, naraża dane na dostęp osoby postronnej.

Projekt rozwiązania: **Automatyczna blokada** odlicza czas od ostatniej aktywności użytkownika i po upływie ustalonego okresu bezczynności porzuca klucz danych oraz odszyfrowany stan, wracając do ekranu blokady. Logikę odliczania wydzielono jako czysty, testowalny wyzwalacz niezależny od warstwy interfejsu, którego metoda odnawiająca jest wywoływana przy każdej aktywności (listing 5.4).

```
function reset(): void {
  anuluj();
  // Po ‘ms‘ bez kolejnego resetu wywołaj callback, w praktyce auto-
     blokadę,
  // która porzuca klucz danych i odszyfrowany stan.
  uchwyt = setTimeout(() => { uchwyt = null; callback(); }, ms);
}
```

Listing 5.4: Wyzwalacz bezczynności porzucający sesję (TypeScript)
Źródło: opracowanie własne

Analiza i ograniczenie: Auto-blokada ogranicza okno czasowe, w którym odblokowana sesja jest narażona na dostęp fizyczny. Jej skuteczność jest jednak ograniczona do porzucenia stanu w pamięci aplikacji, nie chroni przed napastnikiem, który uzyskałby dostęp do zrzutu pamięci procesu w momencie, gdy sesja była aktywna. Stanowi zatem rozsądny kompromis między bezpieczeństwem a wygodą, a nie barierę absolutną.

5.5 Dławienie nieudanych prób

Zagrożenie: Napastnik mający chwilowy dostęp do odblokowywanego urządzenia może próbować oportunistycznie zgadywać hasło, wielokrotnie ponawiając próby odblokowania.

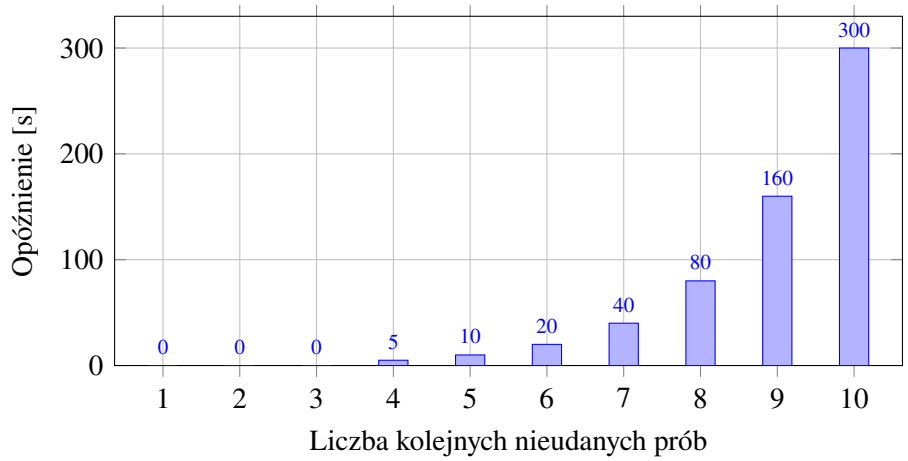
Projekt rozwiązania: Dławienie prób (ang. *throttling*) wprowadza narastające wykładniczo opóźnienie po przekroczeniu progu nieudanych prób, zgodnie z zaleceniem ograniczania tempa prób uwierzytelnienia [11]. Pierwsze próby pozostają bez kary, ponieważ pomyłki się zdarzają, po czym każde kolejne niepowodzenie podwaja opóźnienie aż do ustalonego górnego limitu. Licznik prób przechowywany jest w pamięci trwałej przeglądarki, aby przetrwał odświeżenie strony; nie jest on jednak sekretem, lecz zwykłym licznikiem, więc jego obecność nie narusza zasady braku sekretów w pamięci trwałej. Listing 5.5 przedstawia funkcję opóźnienia.

```
const PROBY_BEZ_KARY = 3;     // pierwsze próby bez opóźnienia
const BAZA_MS = 5_000;        // bazowe opóźnienie po przekroczeniu
   progu
const MAX_MS = 5 * 60 * 1_000; // górny limit (5 minut)

function opoznienieMs(nieudane: number): number {
  if (nieudane <= PROBY_BEZ_KARY) return 0;
  const krok = nieudane - PROBY_BEZ_KARY;       // 1, 2, 3, ...
  return Math.min(MAX_MS, BAZA_MS * 2 ** (krok - 1));
}
```

Listing 5.5: Wykładnicze opóźnienie nieudanych prób (TypeScript)
Źródło: opracowanie własne

Wykładniczy charakter narastania opóźnienia (do górnego limitu 300 s) przedstawia rysunek 5.2.



Rys. 5.2: Narastające wykładniczo opóźnienie po kolejnych nieudanych próbach odblokowania, pierwsze trzy próby bez kary, następnie podwajanie aż do limitu 300 s
Źródło: opracowanie własne

Uczciwe ograniczenie: Dławienie prób jest obroną na żywym interfejsie i **nie** chroni przed atakiem siłowym prowadzonym w trybie offline. Napastnik, który wykradł szyfrotekst, zgaduje hasło własnym skryptem operującym bezpośrednio na funkcji Argon2id, całkowicie pomijając interfejs aplikacji wraz z jego opóźnieniami. Realną barierą offline pozostaje koszt funkcji Argon2id ustalony podczas kalibracji. Mechanizm utrudnia zatem jedynie zgadywanie oportunistyczne na odblokowywanym urządzeniu oraz pełni rolę sygnału ostrzegawczego o trwającym ataku.

5.6 Świadome fokusu czyszczenie schowka

Zagrożenie: Skopiowanie hasła do schowka systemowego naraża je na odczyt przez inne aplikacje lub przez kolejną osobę korzystającą z urządzenia, jeśli pozostanie tam zbyt długo.

Projekt rozwiązania: Po skopiowaniu sekretu uruchamiany jest wspólny, widoczny licznik, po upływie którego schowek jest czyszczony. Rozwiązanie uwzględnia istotne ograniczenie przeglądarek: operacje na schowku wymagają, by karta posiadała fokus. Z tego powodu, jeśli w chwili upływu czasu karta nie ma fokusu, co jest typowe, gdy użytkownik przełączył się do innego okna, aby wkleić hasło, czyszczenie jest odkładane i wykonywane natychmiast po powrocie fokusu, nasłuchując odpowiednich zdarzeń. Dodatkowo schowek jest czyszczony przy każdej blokadzie skarbca. Przed nadpisaniem schowek jest sprawdzany, aby nie usunąć treści, którą użytkownik skopiował w międzyczasie.

Uczciwe ograniczenie: Czyszczenie schowka jest mechanizmem **najlepszego wysiłku**. Ograniczenia przeglądarek oraz systemu operacyjnego, a w szczególności obecność

historii schowka w menedżerach schowka, pozostają poza kontrolą strony internetowej. Rozwiązanie ogranicza okno ekspozycji sekretu, lecz nie gwarantuje jego całkowitego usunięcia ze wszystkich miejsc, w których system mógł go zachować.

5.7 Kalibracja funkcji Argon2id

Ostatnim mechanizmem o charakterze ochronnym jest **kalibracja parametrów Argon2id** na urządzeniu, szczegółowo omówiona w rozdziale 3. Z punktu widzenia ochrony jej rolą jest utrzymanie spójnego, wysokiego kosztu wyprowadzenia klucza mimo różnic w wydajności sprzętu. Mechanizm ten dobiera parametry tak, by czas wyprowadzenia klucza pozostawał w docelowym przedziale, podnosząc najpierw koszt pamięciowy, co odpowiada kolejności strojenia zalecanej w specyfikacji funkcji [3]. Jego uczciwym ograniczeniem jest fakt, że rzeczywista siła ochrony różni się między urządzeniami: na sprzęcie wolniejszym kalibracja dobierze słabsze parametry, aby nie pogorszyć użyteczności.

5.8 Podsumowanie mechanizmów

Tabela 5.2 zestawia omówione mechanizmy wraz z zagrożeniem, na które odpowiadają, oraz ich najważniejszym ograniczeniem. Zestawienie to podkreśla przyjętą w pracy zasadę: każdy mechanizm jest realną, lecz **warstwową** obroną o jasno nazwanych granicach, a nie zabezpieczeniem absolutnym.

Tab. 5.2: Zestawienie zaawansowanych mechanizmów ochrony

Mechanizm	**Zagrożenie**	**Najważniejsze ograniczenie**
Fałszywy skarbiec	wymuszenie podania hasła	nie jest kryptograficzną zaprzeczalnością
Rotacja klucza danych	podejrzenie wycieku klucza	nie chroni przy trwającej kompromitacji
Bramka siły hasła	słabe hasło główne	górne oszacowanie entropii
Auto-blokada	dostęp fizyczny do sesji	tylko stan w pamięci aplikacji
Dławienie prób	zgadywanie na urządzeniu	nie chroni przed atakiem offline
Czyszczenie schowka	ekspozycja sekretu w schowku	najlepszy wysiłek, historia schowka poza kontrolą
Kalibracja Argon2id	słaby koszt KDF na słabym sprzęcie	siła zależna od urządzenia

Źródło: opracowanie własne

Rozdział 6

BEZPIECZEŃSTWO WARSTWY KLIENCKIEJ

W architekturze zerowej wiedzy to przeglądarka, a nie serwer, wykonuje całą kryptografię i przechowuje w pamięci odszyfrowany skarbiec. W konsekwencji warstwa kliencka (ang. *front-end*) staje się pierwszą i zarazem najważniejszą linią obrony, a jej kompromitacja oznacza ujawnienie wszystkich sekretów. Rozdział ten omawia mechanizmy bezpieczeństwa wbudowane w warstwę kliencką: ścisłą politykę bezpieczeństwa treści, rygorystyczne renderowanie treści jako tekstu, nieeksportowalność kluczy, brak sekretów w pamięci trwałej oraz granicę błędów. Rozdział zamyka omówienie dostępności jako elementu jakości i pośrednio bezpieczeństwa.

6.1 Warstwa kliencka jako pierwsza linia obrony

Przeniesienie kryptografii do przeglądarki ma fundamentalną konsekwencję dla modelu zagrożeń: największym ryzykiem przestaje być serwer, a staje się nim wykonanie obcego kodu w kontekście aplikacji. Atak polegający na wstrzyknięciu i uruchomieniu cudzego skryptu (ang. *cross-site scripting*, XSS) [23] jest w tej architekturze równoznaczny z **całkowitą kompromitacją** odblokowanej sesji, napastnik uzyskuje bowiem dostęp do odszyfrowanego skarbca oraz potencjalnie do operacji kryptograficznych. Z tego powodu cała warstwa kliencka została zaprojektowana wokół założenia, że atak XSS należy uczynić maksymalnie trudnym, a jego skutki, ograniczyć. Obrona ma charakter warstwowy: żadna pojedyncza technika nie jest traktowana jako wystarczająca.

6.2 Ścisła polityka bezpieczeństwa treści

Podstawowym narzędziem ograniczania ryzyka XSS jest **polityka bezpieczeństwa treści** (ang. *Content Security Policy*, CSP) [19, 30]. Zastosowano politykę restrykcyjną, opartą na zasadzie odmowy domyślnej: dyrektywa `default-src 'none'` blokuje wszystkie typy zasobów, a każdy potrzebny typ musi zostać włączony jawnie. Dzięki temu nic nie przechodzi „przy okazji”. Listing 6.1 przedstawia treść polityki.

```
default-src 'none';
script-src 'self' 'wasm-unsafe-eval';
style-src 'self';
img-src 'self';
font-src 'self';
connect-src 'self';
base-uri 'none';
form-action 'none';
frame-ancestors 'none';
object-src 'none'
```

Listing 6.1: Polityka bezpieczeństwa treści serwowana przez serwer
Źródło: opracowanie własne

Najistotniejszą dyrektywą jest `script-src 'self' 'wasm-unsafe-eval'`. Wartość `'self'` dopuszcza skrypty wyłącznie z tego samego pochodzenia, całkowicie wykluczając skrypty zdalne oraz wstrzyknięte skrypty wbudowane (brak `'unsafe-inline'`). Towarzysząca jej wartość `'wasm-unsafe-eval'` wymaga osobnego komentarza, ponieważ jej obecność mogłaby budzić wątpliwości. Wartość ta służy **wyłącznie** umożliwieniu przeglądarce skompilowania modułu WebAssembly zawierającego funkcję Argon2id. Nie jest ona równoznaczna z ogólnym `'unsafe-eval'` dla języka JavaScript i nie umożliwia wykonywania łańcuchów znaków jako kodu [30]. Jest to świadomy, wąsko zakresowy kompromis: rezygnacja z niej oznaczałaby brak dostępu do twardej pamięciowo funkcji KDF i konieczność użycia słabszego wariantu zapasowego.

Pozostałe dyrektywy domykają powierzchnię ataku. Style, obrazy, czcionki oraz żądania sieciowe ograniczono do własnego pochodzenia, a dyrektywy `base-uri`, `form-action`, `frame-ancestors` oraz `object-src` zamknięto wartością `'none'`, eliminując odpowiednio przejęcie adresu bazowego, nieautoryzowane wysłanie formularza, osadzanie aplikacji w ramce obcej witryny oraz ładowanie wtyczek. Należy zaznaczyć, że ścisła polityka obejmuje produkcyjne serwowanie zbudowanej aplikacji z tego samego pochodzenia co serwer; deweloperski serwer narzędzia Vite wymaga polityki łagodniejszej i nie jest nią objęty.

6.3 Renderowanie treści jako tekstu

Drugą warstwą obrony przed XSS jest bezwzględne renderowanie treści wpisów jako **tekstu**, nigdy jako kodu HTML. W zastosowanej bibliotece interfejsu domyślny mechanizm osadzania treści automatycznie unieszkodliwia znaki specjalne, jednak istnieje furtka pozwalająca wstawić surowy kod HTML [25]. Furtka ta, mechanizm `dangerouslySetInnerHTML`, została w projekcie **całkowicie zakazana**. W praktyce oznacza to, że nawet gdyby napastnik zdołał umieścić w tytule lub notatce wpisu ciąg wyglądający jak znacznik skryptu, zostałby on wyświetlony jako nieszkodliwy tekst, a nie wykonany. Zakaz ten dotyczy wszystkich

pól pochodzących od użytkownika i jest egzekwowany na poziomie przeglądu kodu jako jedna z żelaznych zasad projektu.

6.4 Nieeksportowalność kluczy

Trzecią warstwą jest ograniczenie dostępności surowego materiału kluczowego dla kodu JavaScript. Klucz opakowujący oraz sesyjny klucz danych przechowywane są jako obiekty kryptograficzne oznaczone jako **nieeksportowalne**, interfejs WebCrypto uniemożliwia wówczas odczytanie ich surowych bajtów z poziomu skryptu [18]. Klucz danych przyjmuje postać eksportowalną wyłącznie na krótką chwilę tworzenia lub zmiany hasła, gdy musi zostać zserializowany na potrzeby opakowania, po czym sesyjnie używana jest jego postać nieeksportowalna. Dodatkowo surowy wynik funkcji Argon2id jest zerowany w pamięci bezpośrednio po zaimportowaniu go do obiektu klucza.

Mechanizm ten nie eliminuje skutków pełnej kompromitacji środowiska, napastnik wykonujący obcy kod mógłby posłużyć się kluczem za pośrednictwem interfejsu WebCrypto, nawet nie znając jego bajtów. Podnosi jednak poprzeczkę: utrudnia eksfiltrację samego klucza, a tym samym ogranicza trwałość ewentualnego wycieku i możliwość jego wykorzystania poza zaatakowaną sesją.

6.5 Brak sekretów w pamięci trwałej

Czwartą warstwą jest konsekwentne nieumieszczanie jakichkolwiek sekretów w pamięci trwałej przeglądarki. Ani klucz, ani odszyfrowany skarbiec nie trafiają do magazynów `localStorage` czy `sessionStorage`; odszyfrowane dane istnieją wyłącznie w ulotnym stanie aplikacji i znikają wraz z zamknięciem karty, blokadą lub odświeżeniem. W pamięci trwałej mogą znaleźć się co najwyżej dane nie będące sekretami: szyfrotekst (który jest bezpieczny), opcjonalny token dostępu, licznik nieudanych prób oraz preferencje interfejsu. Tabela 6.1 porządkuje ten podział.

Tab. 6.1: Co może, a co nie może trafić do pamięci trwałej przeglądarki

Dozwolone (nie-sekrety)	**Zakazane (sekrety)**
Nieprzejrzysty szyfrotekst	Hasło główne
Token dostępu do serwera	Klucz opakowujący oraz klucz danych
Licznik nieudanych prób	Odszyfrowana treść skarbca
Preferencje interfejsu (motyw)	Surowy wynik funkcji KDF

Źródło: opracowanie własne

Podział ten realizuje żelazną zasadę projektu, zgodnie z którą trwałość mogą mieć

wyłącznie dane bezpieczne do ujawnienia. Dzięki temu nawet uzyskanie dostępu do pamięci trwałej przeglądarki nie daje napastnikowi żadnego sekretu.

6.6 Granica błędów jako mechanizm bezpieczeństwa

Piątą warstwą jest **granica błędów** (ang. *error boundary*) [25], mechanizm przechwytujący nieobsłużony błąd renderowania w odblokowanej części aplikacji. Jej rola wykracza poza zwykłą obsługę awarii: zamiast pozostawić użytkownika z uszkodzonym widokiem, w którego pamięci wciąż znajduje się odszyfrowany skarbiec, granica błędów **porzuca klucz danych** i przechodzi w stan blokady, prezentując spokojny ekran awaryjny. Jest to zachowanie bezpieczne wobec awarii (ang. *fail-safe*): w razie nieoczekiwanego błędu system domyślnie wybiera stan bezpieczniejszy. Listing 6.2 przedstawia sedno tego mechanizmu.

```
1 componentDidCatch(): void {
2   // Porzuć klucz danych i odszyfrowane dane, przejście w blokadę
        zamiast
3   // pozostawienia odszyfrowanego skarbca w pamięci uszkodzonego
        widoku.
4   queueMicrotask(() => this.props.onBlad());
5 }
```

Listing 6.2: Granica błędów porzucająca klucz danych przy awarii renderu (TypeScript)
Źródło: opracowanie własne

Mechanizm ten domyka warstwową obronę warstwy klienckiej: nawet nieprzewidziany błąd oprogramowania nie pozostawia odszyfrowanych danych w stanie podwyższonego ryzyka, lecz prowadzi do ich porzucenia.

6.7 Dostępność jako element jakości i bezpieczeństwa

Choć dostępność interfejsu kojarzona jest przede wszystkim z ergonomią, w narzędziu bezpieczeństwa pełni ona również rolę pośrednio związaną z ochroną. Interfejs, który jest przewidywalny i w pełni obsługiwany z klawiatury, ogranicza ryzyko pomyłek użytkownika, na przykład skopiowania niewłaściwej wartości lub przypadkowego ujawnienia sekretu. W projekcie zadbano o widoczny wskaźnik fokusu, pułapkę fokusu oraz odpowiednie atrybuty `aria` w oknach modalnych i palecie poleceń, etykiety dla ikon, a także respektowanie preferencji ograniczenia ruchu.

Element-sygnaturą interfejsu jest spokojny, pełnoekranowy ekran blokady oraz odblokowania, komunikujący stan bezpieczeństwa danych, wraz z miernikiem entropii hasła wyrażonym typograficznie. Zgodnie z przyjętym systemem projektowym powściągliwość interpretowana jest jako wiarygodność: narzędzie bezpieczeństwa ma budzić zaufanie precyzją i spokojem, nie zaś efektownością. Dostępność i spójność wizualna współtworzą zatem wrażenie rzetelności, które w narzędziu tej klasy jest wartością samą w sobie.

Rozdział 7

IMPLEMENTACJA I JAKOŚĆ

Rozdział ten przedstawia praktyczną realizację systemu oraz przyjętą strategię zapewnienia jakości. Omówiono w nim stos technologiczny wraz z uzasadnieniem dokonanych wyborów, kluczowe komponenty aplikacji klienckiej i serwerowej, a także wielowarstwową strategię testów: od testów jednostkowych, przez testy znanych odpowiedzi potwierdzające poprawność prymitywów kryptograficznych, aż po scenariusze całościowe (ang. *end-to-end*) w rzeczywistej przeglądarce. Osobny podrozdział dokumentuje **wyniki wykonania** wszystkich poziomów testów wraz ze zrzutami ekranu z rzeczywistych przebiegów oraz pomiarem pokrycia kodu. Rozdział zamyka omówienie procesu ciągłej integracji oraz audytu łańcucha dostaw.

7.1 Stos technologiczny i jego uzasadnienie

Dobór technologii podporządkowano dwóm nadrzędnym celom: poprawności kryptografii oraz **minimalizacji powierzchni ataku**. W architekturze, w której cały skarbiec żyje w przeglądarce, każda dodatkowa zależność jest potencjalnym wektorem ataku łańcucha dostaw, dlatego liczbę zależności świadomie ograniczono, a każdą z nich uzasadniono. Tabela 7.1 zestawia najważniejsze elementy stosu.

Tab. 7.1: Stos technologiczny projektu

Warstwa	Technologie
Warstwa kliencka	React, TypeScript (tryb ścisły), Vite, WebCrypto, `hash-wasm` (Argon2id)
Warstwa serwerowa	FastAPI, Uvicorn, Pydantic (`extra="forbid"`), SQLite, Python
Testy i jakość	Vitest, `@testing-library/react`, happy-dom, pomiar pokrycia, puppeteer-core (testy całościowe)
CI i łańcuch dostaw	GitHub Actions, `npm audit`, `pip-audit`, Dependabot, akcje przypięte do skrótów SHA

Źródło: opracowanie własne

Wybór języka TypeScript w trybie ścisłym [28] wynika z dążenia do wczesnego wykrywania błędów: silny system typów eliminuje całą klasę pomyłek, zanim kod trafi do

przeglądarki. Warstwę widoku zbudowano na bibliotece React [25]. Po stronie kryptografii filarem jest interfejs WebCrypto [18], dostępny natywnie i nie wymagający żadnej zależności; jedynym istotnym wyjątkiem jest biblioteka dostarczająca funkcję Argon2id w postaci modułu WebAssembly, której użycie uzasadniono w rozdziale 3 brakiem tej funkcji w WebCrypto. Po stronie serwera technologia FastAPI wraz z biblioteką Pydantic [8] zapewnia zwięzłą definicję i twardą walidację schematu, co bezpośrednio realizuje wymaganie odrzucania obcych pól.

7.2 Kluczowe komponenty

Aplikację kliencką podzielono na wyraźnie wyodrębnione moduły o jednoznacznych odpowiedzialnościach. Rdzeń kryptograficzny skupia operacje na kluczach i szyfrowaniu, realizując model KEK/DEK oraz wszystkie funkcje z nim związane: tworzenie i odblokowanie skarbca, zmianę hasła, rotację klucza danych oraz obsługę slotu przynęty. Warstwa komunikacji odpowiada za wymianę pakietu z serwerem, w tym za obsługę kontroli współbieżności. Moduły funkcjonalne, generator haseł, audyt zdrowia haseł, wyliczanie kodów jednorazowych, schowek, zostały oddzielone od warstwy prezentacji, dzięki czemu ich logika jest testowalna w izolacji. Powłoka interfejsu zapewnia paletę poleceń, obsługę klawiaturą oraz granicę błędów.

Po stronie serwera architektura jest celowo płaska i minimalna: moduł tras definiuje dwa punkty końcowe, moduł schematów opisuje format pakietu wraz z regułą odrzucania obcych pól, moduł magazynu utrwala pakiet w bazie wraz z licznikiem rewizji, a moduł nagłówków bezpieczeństwa dokleja do każdej odpowiedzi politykę bezpieczeństwa treści oraz pozostałe nagłówki. Ten rozdział odpowiedzialności sprawia, że logika serwera jest na tyle prosta, iż łatwo zweryfikować jej zgodność z zasadą pasywnego magazynu danych.

7.3 Strategia testów

Jakość rozwiązania potwierdzono wielowarstwową strategią testów, obejmującą sto dwadzieścia cztery testy po stronie klienta oraz dziewiętnaście po stronie serwera. Wyróżniono trzy uzupełniające się poziomy, zestawione w tabeli 7.2; rezultaty ich rzeczywistego wykonania przedstawiono w podrozdziale 7.4.

Tab. 7.2: Poziomy strategii testów

Poziom	**Cel**
Testy jednostkowe	Weryfikacja logiki modułów w izolacji (entropia, dławienie prób, schowek, generator)
Testy znanych odpowiedzi	Dowód zgodności prymitywów kryptograficznych ze standardami NIST/RFC
Testy całościowe	Weryfikacja zachowania całej aplikacji w rzeczywistej przeglądarce

Źródło: opracowanie własne

7.3.1 Testy jednostkowe oraz testy odwracalności

Najliczniejszą grupę stanowią testy jednostkowe weryfikujące logikę poszczególnych modułów. W warstwie kryptograficznej kluczowe są testy odwracalności (ang. *round-trip*) oraz testy odmowy: potwierdzają one, że dane zaszyfrowane prawidłowym hasłem dają się odszyfrować, że błędne hasło prowadzi do jawnej odmowy, a modyfikacja choćby jednego bajtu szyfrotekstu jest wykrywana jako naruszenie integralności. Testy te odpowiadają wprost wymaganiom postawionym w konwencjach projektu.

7.3.2 Testy znanych odpowiedzi

Szczególne znaczenie dla wiarygodności pracy mają **testy znanych odpowiedzi** (ang. *known-answer tests*, KAT). W odróżnieniu od testów odwracalności, które sprawdzają jedynie wzajemną odwracalność operacji, testy KAT porównują wynik implementacji z **oficjalnymi wektorami** pochodzącymi ze standardów NIST oraz RFC [6, 17, 26]. Stanowi to znacznie mocniejszy dowód poprawności: test odwracalności nie wykryłby na przykład użycia niewłaściwego trybu czy parametru, o ile tylko operacja pozostawałaby odwracalna, podczas gdy test KAT natychmiast ujawniłby rozbieżność z wzorcem. Listing 7.1 przedstawia przykład takiego testu dla funkcji PBKDF2.

```
// Porównanie z oficjalnym wektorem: P="password", S="salt", c=1, dlugosc 32 B.
const k = await subtle.importKey("raw", enc("password"), "PBKDF2", false,
                                 ["deriveBits"]);
const bits = await subtle.deriveBits(
  { name: "PBKDF2", salt: enc("salt"), iterations: 1, hash: "SHA-256" }, k, 256);
expect(hex(bits)).toBe(
  "120fb6cffcf8b32c43e7225256c4f837a86548c92ccc35480805987cb70be17b");
```

Listing 7.1: Test znanej odpowiedzi dla PBKDF2 względem wektora standardowego (TypeScript)
Źródło: opracowanie własne

Testami KAT objęto funkcję PBKDF2, szyfr AES-256-GCM oraz algorytm opakowania kluczy AES-KW. Uzupełnia je test wykorzystujący „zamrożony" pakiet danych, szyfrotekst zbudowany niezależnie, którego poprawne odszyfrowanie dowodzi stabilności i interoperacyjności formatu w czasie. Dzięki temu każda przypadkowa zmiana formatu lub regresja prymitywu zostanie natychmiast wykryta.

7.3.3 Testy całościowe

Najwyższy poziom stanowią testy całościowe uruchamiane w rzeczywistej przeglądarce. Weryfikują one zachowanie całej aplikacji w scenariuszach zbliżonych do użycia rzeczywistego, obejmując między innymi dowód braku hasła w ruchu sieciowym, kontrolę współbieżności, działanie fałszywego skarbca, czyszczenie schowka, rotację klucza danych oraz obsługę kosza. Szczególnie istotny jest scenariusz nagrywający żądania sieciowe i weryfikujący, że hasło główne nie pojawia się w żadnym z nich, jest to automatyczna, wykonywalna weryfikacja najważniejszej żelaznej zasady projektu.

7.4 Wyniki wykonania testów

Przedstawiona wyżej strategia nie pozostaje deklaracją projektową: wszystkie trzy poziomy testów zostały wykonane, a niniejszy podrozdział dokumentuje rezultaty tych uruchomień. Zestawy uruchomiono na urządzeniu referencyjnym, tym samym, na którym wykonano pomiary kosztu funkcji Argon2id przedstawione w rozdziale 3; jego konfigurację zestawia tabela 7.3. Wszystkie zamieszczone dalej rysunki są zrzutami ekranu z rzeczywistych przebiegów.

Tab. 7.3: Środowisko wykonania testów (urządzenie referencyjne)

Element	**Konfiguracja**
Procesor i pamięć	Intel Core i7-7500U (4 wątki, 2,70 GHz), 16 GiB RAM
System operacyjny	Linux, jądro 6.19, dystrybucja oparta na Debianie
Warstwa kliencka	Node.js 20.19, Vitest 2.1.9, happy-dom
Warstwa serwerowa	Python 3.13, pytest 9.0.3
Przeglądarka (testy całościowe)	Chromium 148, sterowany przez puppeteer-core

Źródło: opracowanie własne

7.4.1 Wyniki testów warstwy klienckiej

Zestaw kliencki, uruchamiany poleceniem `npm test`, obejmuje 20 plików testowych zawierających łącznie **124 przypadki testowe**. Wszystkie zakończyły się powodzeniem, a pełny przebieg trwał poniżej ośmiu sekund (rysunek 7.1). Tabela 7.4 porządkuje ten sam wynik według obszarów funkcjonalnych, uwidaczniając, że najliczniej obsadzone są rdzeń kryptograficzny oraz moduły realizujące mechanizmy ochrony.

```
$ npm test          (frontend — Vitest)

 RUN  v2.1.9 /home/mrobot/Documents/Password_Manager/frontend

 ✓ src/model.test.ts (10 tests) 30ms
 ✓ src/crypto/kat.test.ts (6 tests) 38ms
 ✓ src/api/skarbiec.test.ts (5 tests) 120ms
 ✓ src/przeplyw.test.ts (3 tests) 231ms
 ✓ src/audyt/analiza.test.ts (5 tests) 9ms
 ✓ src/totp.test.ts (14 tests) 33ms
 ✓ src/skarbiec/generator.test.ts (10 tests) 19ms
 ✓ src/crypto/argon2.test.ts (4 tests) 355ms
 ✓ src/schowek.test.ts (5 tests) 49ms
 ✓ src/blokadaProb.test.ts (5 tests) 11ms
 ✓ src/powloka/GranicaBledow.test.tsx (3 tests) 23ms
 ✓ src/bezczynnosc.test.ts (3 tests) 15ms
 ✓ src/crypto/skarbiec.test.ts (18 tests) 3301ms
 ✓ src/entropia.test.ts (5 tests) 9ms
 ✓ src/kopia.test.ts (5 tests) 10ms
 ✓ src/skarbiec/ListaWpisow.test.tsx (9 tests) 243ms
 ✓ src/powloka/PaletaPolecen.test.tsx (4 tests) 120ms
 ✓ src/ekran/EkranBlokady.test.tsx (6 tests) 226ms
 ✓ src/powloka/StanPusty.test.tsx (2 tests) 40ms
 ✓ src/ekran/MiernikEntropii.test.tsx (2 tests) 52ms

 Test Files  20 passed (20)
      Tests  124 passed (124)
   Start at  17:49:41
   Duration  7.67s (transform 873ms, setup 0ms, collect 2.99s, tests 4.93s, environment 3.66s, prepare 2.59s)
```

Rys. 7.1: Zrzut ekranu z wykonania zestawu testów warstwy klienckiej: 20 plików testowych, 124 przypadki, wszystkie zaliczone

Źródło: opracowanie własne (zrzut ekranu)

Tab. 7.4: Wyniki testów warstwy klienckiej według obszarów

Obszar	Testów	Zalicz.	Wynik
Rdzeń kryptograficzny (tworzenie, odblokowanie, zmiana hasła, rotacja, slot przynęty)	22	22	bez błędów
Testy znanych odpowiedzi wobec wektorów NIST/RFC	6	6	bez błędów
Model danych oraz kopia zapasowa	15	15	bez błędów
Komunikacja z serwerem i kontrola współbieżności	8	8	bez błędów
Mechanizmy ochrony (entropia, dławienie prób, bezczynność, schowek)	18	18	bez błędów
Moduły funkcjonalne (generator, audyt haseł, kody jednorazowe)	29	29	bez błędów
Warstwa prezentacji i powłoka interfejsu	26	26	bez błędów
Razem	**124**	**124**	**100%**

Źródło: opracowanie własne na podstawie przebiegu z rysunku 7.1

7.4.2 Wyniki testów warstwy serwerowej

Zestaw serwerowy liczy **19 przypadków testowych** w czterech plikach i również w całości kończy się powodzeniem (rysunek 7.2). Weryfikuje on przede wszystkim te własności, które czynią serwer pasywnym magazynem danych: odrzucanie pól spoza zdefiniowanego formatu (w tym próby dosłania pola z hasłem), brak jakiegokolwiek punktu końcowego logowania, obecność nagłówków bezpieczeństwa oraz poprawność kontroli współbieżności. Zestawienie według plików zawiera tabela 7.5.

```
$ python -m pytest -v   (backend — pytest)

============================= test session starts ==============================
platform linux -- Python 3.13.12, pytest-9.0.3, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/mrobot/Documents/Password_Manager/backend
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.12.1, typeguard-4.4.4
collecting ... collected 19 items

tests/test_dostep.py::test_bez_tokenu_401 PASSED                         [  5%]
tests/test_dostep.py::test_zly_token_401 PASSED                          [ 10%]
tests/test_dostep.py::test_poprawny_token_dziala PASSED                  [ 15%]
tests/test_dostep.py::test_zdrowie_nie_wymaga_tokenu PASSED              [ 21%]
tests/test_przyneta.py::test_przechowuje_i_oddaje_przynete PASSED        [ 26%]
tests/test_przyneta.py::test_bez_przynety_brak_klucza_null PASSED        [ 31%]
tests/test_przyneta.py::test_odrzuca_niepelny_slot_przynety PASSED       [ 36%]
tests/test_przyneta.py::test_odrzuca_obce_pole_w_przynecie PASSED        [ 42%]
tests/test_skarbiec.py::test_get_pusty_zwraca_404 PASSED                 [ 47%]
tests/test_skarbiec.py::test_put_a_potem_get_zwraca_to_samo PASSED       [ 52%]
tests/test_skarbiec.py::test_put_nadpisuje_poprzedni_skarbiec PASSED     [ 57%]
tests/test_skarbiec.py::test_odrzuca_dodatkowe_pole_np_haslo PASSED      [ 63%]
tests/test_skarbiec.py::test_odrzuca_brak_wymaganego_pola PASSED         [ 68%]
tests/test_skarbiec.py::test_naglowki_bezpieczenstwa_obecne PASSED       [ 73%]
tests/test_skarbiec.py::test_brak_endpointu_logowania PASSED             [ 78%]
tests/test_wspolbieznosc.py::test_get_zwraca_etag_rosnacy PASSED         [ 84%]
tests/test_wspolbieznosc.py::test_put_z_aktualnym_if_match_przechodzi PASSED [ 89%]
tests/test_wspolbieznosc.py::test_put_z_nieaktualnym_if_match_konflikt PASSED [ 94%]
tests/test_wspolbieznosc.py::test_put_bez_if_match_jest_bezwarunkowy PASSED [100%]

============================== 19 passed in 0.84s ==============================
```

Rys. 7.2: Zrzut ekranu z wykonania zestawu testów warstwy serwerowej: 19 przypadków, wszystkie zaliczone

Źródło: opracowanie własne (zrzut ekranu)

Tab. 7.5: Wyniki testów warstwy serwerowej

Plik	Zakres weryfikacji	Testów	Zalicz.
`test_skarbiec`	przechowywanie i zastępowanie pakietu, odrzucanie obcych pól, nagłówki bezpieczeństwa, brak punktu logowania	7	7
`test_wspolbieznosc`	znaczniki `ETag` oraz `If-Match`, konflikt `412` zamiast cichego nadpisania	4	4
`test_przyneta`	przechowywanie slotu przynęty i walidacja jego kształtu	4	4
`test_dostep`	opcjonalny token dostępu oraz punkt kontroli sprawności	4	4
Razem		**19**	**19**

Źródło: opracowanie własne na podstawie przebiegu z rysunku 7.2

7.4.3 Pokrycie kodu

Sam fakt zaliczenia testów nie mówi jeszcze, jaka część kodu została nimi faktycznie wykonana. Odpowiedzi na to pytanie dostarcza pomiar **pokrycia kodu** przeprowadzony narzędziem `v8` (rysunek 7.3), którego wyniki zbiorcze zestawiono w tabeli 7.6. Pomiar obejmuje logiczne jądro aplikacji, moduły kryptograficzne, komunikacyjne, model danych oraz mechanizmy ochrony, czyli tę część kodu, której poprawność decyduje o bezpieczeństwie.

```
$ npm run test:coverage   (pokrycie kodu — v8)

 % Coverage report from v8
-----------------|---------|----------|---------|---------|--------------------
File             | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----------------|---------|----------|---------|---------|--------------------
All files        |    96.1 |    87.45 |     100 |    96.1 |
 src             |   97.42 |     87.7 |     100 |   97.42 |
  bezczynnosc.ts |     100 |      100 |     100 |     100 |
  blokadaProb.ts |   92.68 |    85.71 |     100 |   92.68 | 50-52
  entropia.ts    |   93.93 |    94.44 |     100 |   93.93 | 35-36
  model.ts       |     100 |    88.23 |     100 |     100 | ...-119,146-147,151
  totp.ts        |   97.84 |    82.35 |     100 |   97.84 | 115-116
 src/api         |    93.1 |    86.36 |     100 |    93.1 |
  skarbiec.ts    |    93.1 |    86.36 |     100 |    93.1 | 58-59,87-88
 src/audyt       |     100 |    93.33 |     100 |     100 |
  analiza.ts     |     100 |    93.33 |     100 |     100 | 71
 src/crypto      |   95.49 |       85 |     100 |   95.49 |
  argon2.ts      |   97.14 |    66.66 |     100 |   97.14 | 136-137
  bledy.ts       |     100 |      100 |     100 |     100 |
  index.ts       |     100 |      100 |     100 |     100 |
  kodowanie.ts   |     100 |      100 |     100 |     100 |
  pbkdf2.ts      |   92.85 |       80 |     100 |   92.85 | 24-25
  skarbiec.ts    |   94.67 |    88.46 |     100 |   94.67 | ...,260-261,284-285
  typy.ts        |     100 |      100 |     100 |     100 |
 src/skarbiec    |   94.59 |    91.66 |     100 |   94.59 |
  generator.ts   |   94.59 |    91.66 |     100 |   94.59 | 61-62,106-107
-----------------|---------|----------|---------|---------|--------------------

============================== Coverage summary ==============================
Statements   : 96.1% ( 790/822 )
Branches     : 87.45% ( 230/263 )
Functions    : 100% ( 69/69 )
Lines        : 96.1% ( 790/822 )
==============================================================================
```

Rys. 7.3: Zrzut ekranu z pomiaru pokrycia kodu logicznego jądra aplikacji
Źródło: opracowanie własne (zrzut ekranu)

Tab. 7.6: Pokrycie kodu logicznego jądra aplikacji

Miara	Pokrycie	Pokryte / wszystkie
Instrukcje	96,10%	790 / 822
Gałęzie	87,45%	230 / 263
Funkcje	100,00%	69 / 69
Linie	96,10%	790 / 822

Źródło: opracowanie własne na podstawie przebiegu z rysunku 7.3

Stuprocentowe pokrycie funkcji oznacza, że żadna funkcja jądra nie pozostaje nieprzetestowana. Niepokryte fragmenty instrukcji i gałęzi to w większości ścieżki obsługi sytuacji wyjątkowych, na przykład reakcja na uszkodzony wpis w pamięci trwałej przeglądarki lub na niedostępność modułu WebAssembly, oraz nieliczne gałęzie logiki wykonywane wyłącznie przy szczególnych parametrach, jak kolejne kroki kalibracji funkcji Argon2id.

Osiągnięty poziom z zapasem przekracza progi egzekwowane mechanicznie w procesie ciągłej integracji, opisane w podrozdziale 7.5.

7.4.4 Wyniki testów całościowych

Najmocniejszym dowodem poprawności całego rozwiązania są testy całościowe, wykonywane w rzeczywistej przeglądarce sterowanej programowo. Uruchomiono **20 niezależnych scenariuszy**, każdy na świeżo wystartowanym serwerze i pustej bazie danych; wszystkie zakończyły się powodzeniem (rysunek 7.5), a ich zakres zestawia tabela 7.7.

Szczególne znaczenie ma scenariusz pierwszy, stanowiący wykonywalny dowód najważniejszej żelaznej zasady projektu. Sterownik nagrywa *cały* ruch sieciowy przeglądarki i przeszukuje treść każdego żądania pod kątem hasła głównego. Jak pokazuje rysunek 7.4, spośród 152 żądań zarejestrowanych podczas pełnego cyklu utworzenia i odblokowania skarbca pięć trafiło do punktu końcowego `/skarbiec`, a liczba żądań zawierających hasło główne wyniosła **zero**. Żądania odczytu mają puste ciało, natomiast jedyne żądanie zapisu przenosi wyłącznie pakiet danych z metadanymi funkcji KDF i szyfrotekstem.

```
$ npm run e2e -- odblokowanie   (Chromium, puppeteer)
──────── e2e: odblokowanie ────────

=== Ruch sieciowy do /skarbiec ===
  GET  /skarbiec   cialo=(brak)
  GET  /skarbiec   cialo=(brak)
  PUT  /skarbiec   cialo={"wersja":1,"kdf":{"nazwa":"argon2id","sol":"Zc2QrWEtrpjAHnJ…
  GET  /skarbiec   cialo=(brak)
  GET  /skarbiec   cialo=(brak)

Żądań ogółem: 152, do /skarbiec: 5
Żądań zawierających hasło główne: 0

OK: hasło główne nie opuściło przeglądarki. Zrzuty w: e2e/zrzuty
  ✓ odblokowanie
```

Rys. 7.4: Zrzut ekranu z wykonywalnego dowodu, że hasło główne nie opuszcza przeglądarki: zero żądań zawierających hasło spośród 152 nagranych

Źródło: opracowanie własne (zrzut ekranu)

```
$ npm run e2e   (wiersze podsumowujące 20 scenariuszy)

  ✓ odblokowanie
  ✓ skarbiec
  ✓ y1-kopia
  ✓ r2-proby
  ✓ faza-a
  ✓ faza-b
  ✓ faza-c
  ✓ faza-d
  ✓ faza-y1
  ✓ faza-y2
  ✓ faza-r2
  ✓ faza-y4
  ✓ faza-duress
  ✓ faza-schowek
  ✓ faza-rotacja
  ✓ faza-kosz
  ✓ faza-otpauth
  ✓ faza-klawiatura
  ✓ faza-szybkie
  ✓ faza-zaznaczanie
======== Razem: 20 driverów, porażki: 0 ========
```

Rys. 7.5: Zrzut ekranu z wierszami podsumowującymi wszystkie 20 scenariuszy całościowych, brak porażek

Źródło: opracowanie własne (zrzut ekranu)

Tab. 7.7: Scenariusze testów całościowych i ich wyniki

Weryfikowana własność	**Wynik**
Brak hasła głównego w ruchu sieciowym (dowód nagrywany)	zaliczony
Pełny przepływ skarbca; treść wpisów renderowana jako tekst (XSS)	zaliczony
Eksport i import kopii zapasowej odtwarza skarbiec	zaliczony
Dławienie nieudanych prób odblokowania	zaliczony
Powłoka interfejsu, dwupanelowy układ listy i szczegółów, licznik auto-blokady	zaliczony
Audyt zdrowia haseł, wyszukiwarka, znaczniki i ulubione	zaliczony
Dziennik zdarzeń, generator haseł, ponowna kalibracja KDF	zaliczony
Paleta poleceń, skróty klawiaturowe, skok z audytu do wpisu	zaliczony
Wariant zapasowy PBKDF2 przy niedostępnym WebAssembly	zaliczony
Znaczniki czasu oraz audyt „stare hasła”	zaliczony
Kontrola współbieżności chroni przed cichym nadpisaniem	zaliczony
Kody jednorazowe TOTP wyliczane lokalnie	zaliczony
Fałszywy skarbiec otwierany pod przymusem	zaliczony
Czyszczenie schowka wraz ze wspólnym licznikiem	zaliczony
Rotacja klucza danych, ciągłość zapisu i odblokowania	zaliczony
Kosz: usuwanie miękkie, przywracanie i usuwanie trwałe	zaliczony
Import sekretu jednorazowego z odsyłacza `otpauth://`	zaliczony
Nawigacja klawiaturą po liście wpisów	zaliczony
Szybkie kopiowanie loginu i hasła z wiersza listy	zaliczony
Zaznaczanie wielu wpisów, masowe znaczniki i przenoszenie do kosza	zaliczony
Razem 20 scenariuszy	**20 zaliczonych**

Źródło: opracowanie własne na podstawie przebiegu z rysunku 7.5

Każdy scenariusz zapisuje ponadto zrzuty ekranu badanych widoków aplikacji. Wy-

brane z nich, dokumentujące działanie kolejnych mechanizmów ochrony, zamieszczono w dodatku D.

7.4.5 Podsumowanie wyników

Zbiorcze zestawienie wszystkich poziomów weryfikacji zawiera tabela 7.8. Łącznie wykonano 143 przypadki testowe warstwy klienckiej i serwerowej oraz 20 scenariuszy całościowych, co daje 163 niezależne przebiegi weryfikacyjne, bez ani jednego niepowodzenia. Wynik ten pozwala uznać deklarowaną w pracy *weryfikowalność* warstwy kryptograficznej za własność potwierdzoną empirycznie, a nie jedynie postulowaną.

Tab. 7.8: Zbiorcze wyniki wykonania wszystkich poziomów testów

Poziom	Wykonanych	Zalicz.	Czas przebiegu
Testy warstwy klienckiej	124	124	7,7 s
Testy warstwy serwerowej	19	19	0,8 s
Testy całościowe (przeglądarka)	20	20	177 s
Razem	**163**	**163**	**brak porażek**

Źródło: opracowanie własne

7.5 Ciągła integracja i audyt łańcucha dostaw

Wszystkie poziomy testów spięto w procesie **ciągłej integracji** realizowanym za pomocą mechanizmu GitHub Actions. Każda zmiana w kodzie uruchamia pełen zestaw bramek: sprawdzenie typów, wykonanie testów, weryfikację progu pokrycia kodu, budowę aplikacji oraz testy całościowe. Progi pokrycia logicznego jądra ustalono na poziomie dziewięćdziesięciu procent dla instrukcji i linii, dziewięćdziesięciu pięciu procent dla funkcji oraz osiemdziesięciu procent dla gałęzi, co czyni je egzekwowalną bramką jakości, spadek poniżej któregokolwiek z progów zatrzymuje proces integracji.

Odrębną, istotną bramką jest **audyt łańcucha dostaw** [23]. Skoro każda zależność stanowi powierzchnię ataku, proces integracji obejmuje automatyczne skanowanie zależności narzędziami `npm audit` oraz `pip-audit`, a ich aktualność nadzoruje mechanizm Dependabot. Dodatkowo użyte akcje przypięto do konkretnych skrótów SHA, co chroni przed cichym podmienieniem ich zawartości. Zestaw tych środków sprawia, że wprowadzenie znanej podatności poprzez zależność lub regresja prymitywu kryptograficznego skutkuje natychmiastowym zatrzymaniem procesu, jakość i bezpieczeństwo są więc egzekwowane mechanicznie, a nie pozostawione dyscyplinie.

Rozdział 8

WERYFIKACJA, OGRANICZENIA I KIERUNKI ROZWOJU

Rozdział ten domyka część merytoryczną pracy. Przedstawiono w nim wyniki weryfikacji rozwiązania, zarówno dowody poprawności kryptograficznej, jak i rezultaty pomiarów wydajności, następnie w sposób uczciwy i wyczerpujący omówiono ograniczenia systemu, a na końcu wskazano kierunki dalszego rozwoju. Świadome i jawne nazwanie ograniczeń traktowane jest tu jako element rzetelności naukowej, nie zaś jako przyznanie się do niedoskonałości.

8.1 Weryfikacja poprawności i wydajności

Poprawność warstwy kryptograficznej potwierdzono opisanymi w rozdziale 7 testami znanych odpowiedzi względem wektorów NIST oraz RFC, uzupełnionymi testami odwracalności i testami odmowy. Łącznie dowodzą one, że zastosowane prymitywy są zgodne ze standardami, że format pakietu jest stabilny, a mechanizmy wykrywania błędnego hasła oraz naruszonej integralności działają zgodnie z założeniami. Najważniejsza żelazna zasada, brak hasła w ruchu sieciowym, podlega automatycznej weryfikacji w scenariuszu całościowym nagrywającym żądania. Wszystkie wymienione testy zostały wykonane, a ich rezultaty, 124 przypadki po stronie klienta, 19 po stronie serwera oraz 20 scenariuszy całościowych, wszystkie zaliczone, przy pokryciu logicznego jądra aplikacji na poziomie 96,1%, udokumentowano zrzutami ekranu i zestawieniami w podrozdziale 7.4.

Wydajność funkcji wyprowadzania klucza zmierzono na urządzeniu referencyjnym, a jej wyniki przedstawiono w rozdziale 3 (tabela 3.1). Potwierdzają one, że dobrane przez kalibrację parametry utrzymują czas wyprowadzenia klucza w docelowym przedziale rzędu połowy do jednej sekundy, co stanowi rozsądny kompromis między kosztem ataku siłowego a komfortem użytkownika. Zestawienie dowodów weryfikacji zawiera tabela 8.1.

Tab. 8.1: Zestawienie dowodów weryfikacji

Własność	**Sposób weryfikacji**
Zgodność prymitywów	Testy znanych odpowiedzi (NIST/RFC)
Stabilność formatu	„Zamrożony" pakiet danych (test interoperacyjności)
Odwracalność szyfrowania	Testy odwracalności (szyfrowanie–deszyfrowanie)
Wykrywanie naruszeń	Testy odmowy (błędne hasło, zmieniony bajt)
Brak hasła w sieci	Scenariusz całościowy nagrywający żądania
Koszt funkcji KDF	Pomiar czasu wykonania (kalibracja, pomiar wzorcowy)

Źródło: opracowanie własne

8.2 Uczciwe ograniczenia

Rzetelna ocena rozwiązania wymaga jawnego nazwania jego granic. Poniżej zebrano najważniejsze ograniczenia, z których część omówiono już przy poszczególnych mechanizmach, tutaj zaś ujęto je całościowo.

Zasięg lokalny i jeden użytkownik: System obsługuje pojedynczego użytkownika oraz pojedynczy skarbiec, bez automatycznej synchronizacji wielu urządzeń poza ręcznym eksportem i importem kopii. Jest to świadome ograniczenie zakresu, nie zaś przeoczenie.

Granica zaprzeczalności fałszywego skarbca: Jak szczegółowo omówiono w rozdziale 5, fałszywy skarbiec chroni przed wymuszeniem, lecz nie zapewnia kryptograficznej zaprzeczalności, analiza surowego pakietu przy znajomości formatu ujawnia obecność drugiego slotu. Prawdziwa nieodróżnialność wymagałaby znacznie bardziej złożonej konstrukcji ukrytego woluminu.

Ograniczona gwarancja czyszczenia schowka: Czyszczenie schowka podlega ograniczeniom przeglądarek i systemu operacyjnego; historia schowka utrzymywana przez system pozostaje poza kontrolą aplikacji.

Nieusuwalne ryzyko XSS: W architekturze webowej skuteczny atak XSS oznacza pełną kompromitację odblokowanej sesji. Ryzyko to jest mitygowane warstwowo (ścisła polityka treści, renderowanie jako tekst, brak sekretów w pamięci trwałej, nieeksportowalność kluczy), lecz nie daje się całkowicie wyeliminować, jest bowiem wpisane w naturę platformy.

Słabość wariantu zapasowego: Wariant zapasowy PBKDF2 jest słabszy od Argon2id, ponieważ nie jest funkcją twardą pamięciowo. Stanowi kompromis na rzecz działania w środowiskach pozbawionych WebAssembly.

Zależność siły ochrony od urządzenia: Parametry Argon2id dobierane są przez kalibrację, przez co rzeczywista siła ochrony różni się między maszynami: na sprzęcie wolniejszym dobierane są słabsze parametry.

Brak formalnego audytu i ograniczenia TOTP: Projekt ma charakter akademicki i nie przeszedł formalnego audytu bezpieczeństwa. Wyliczanie kodów jednorazowych obsługuje wyłącznie wariant domyślny standardu [20]. Ponadto w pamięci trwałej przeglądarki przechowywane są jawne dane nie będące sekretami (token dostępu, licznik prób, preferencje), co jest udokumentowane.

8.3 Kierunki rozwoju

Zidentyfikowane ograniczenia wyznaczają naturalne kierunki dalszych prac. Najbardziej oczekiwanym rozszerzeniem jest **synchronizacja wielu urządzeń** zachowująca zasadę zerowej wiedzy, architektura klient–serwer została zaprojektowana z myślą o takiej rozbudowie, a kontrola współbieżności stanowi jej pierwszy element. Drugim kierunkiem jest **rozszerzenie przeglądarki** wraz z mechanizmem autouzupełniania, które wyeliminowałoby ręczne kopiowanie poświadczeń, zmniejszając zarazem ekspozycję sekretów w schowku.

W obszarze ochrony przed wymuszeniem wartościowym, choć złożonym kierunkiem jest implementacja **ukrytego woluminu** zapewniającego prawdziwą zaprzeczalność. Wzmocnieniu uwierzytelniania służyłoby z kolei wsparcie dla **sprzętowych kluczy bezpieczeństwa** oraz standardu WebAuthn [31], a podniesieniu wiarygodności całego rozwiązania, poddanie go **formalnemu audytowi bezpieczeństwa**. Kierunki te zestawiono w tabeli 8.2.

Tab. 8.2: Kierunki dalszego rozwoju

Kierunek	**Korzyść**
Synchronizacja wielu urządzeń	Wygoda przy zachowaniu zasady zerowej wiedzy
Rozszerzenie przeglądarki	Autouzupełnianie, mniejsza ekspozycja schowka
Ukryty wolumin	Prawdziwa kryptograficzna zaprzeczalność
Klucze sprzętowe i WebAuthn	Silniejsze uwierzytelnianie, odporne na wyłudzanie poświadczeń
Formalny audyt bezpieczeństwa	Niezależne potwierdzenie poprawności

Źródło: opracowanie własne

Wskazane kierunki nie podważają wartości obecnego rozwiązania, lecz wytyczają drogę jego dojrzewania od akademickiego prototypu ku narzędziu gotowemu do zastosowań praktycznych. Wspólnym mianownikiem wszystkich proponowanych rozszerzeń jest zachowanie żelaznych zasad projektu, w szczególności tej, że hasło główne nigdy nie opuszcza urządzenia użytkownika.

ZAKOŃCZENIE

Celem niniejszej pracy było zaprojektowanie, zaimplementowanie oraz weryfikacja lokalnego menedżera haseł działającego w architekturze zerowej wiedzy, w którym serwer jest fizycznie niezdolny do poznania hasła głównego ani kluczy użytkownika, a który ponadto wyposażono w zestaw zaawansowanych mechanizmów ochrony. Cel ten został zrealizowany: powstał kompletny, działający prototyp, którego poprawność kryptograficzną potwierdzono testami, a architekturę i mechanizmy ochrony omówiono i poddano analizie.

W toku pracy zrealizowano kolejno wszystkie cele szczegółowe. W rozdziałach pierwszym i drugim wprowadzono w problematykę zarządzania hasłami, porównano istniejące rozwiązania oraz zbudowano na ich tle model zagrożeń, z którego wyprowadzono wymagania stawiane własnemu projektowi. Rozdział trzeci uzasadnił dobór współczesnych prymitywów kryptograficznych, funkcji Argon2id, szyfru AES-256-GCM oraz algorytmu opakowania kluczy AES-KW, spiętych dwupoziomowym modelem KEK/DEK. Rozdział czwarty przedstawił architekturę systemu z serwerem w roli pasywnego magazynu danych oraz formatem pakietu danych obejmującym opcjonalny drugi slot. Rozdział piąty, stanowiący rdzeń pracy, szczegółowo omówił zaawansowane mechanizmy ochrony z fałszywym skarbcem otwieranym pod przymusem na czele. Rozdział szósty poświęcono bezpieczeństwu warstwy klienckiej jako pierwszej linii obrony, a rozdział siódmy dotyczy implementacji oraz wielowarstwowej strategii testów. Rozdział ósmy zebrał wyniki weryfikacji, ograniczenia oraz kierunki rozwoju.

Założenia projektowe przyjęte we wstępie zostały w pełni zrealizowane. Wykazano w praktyce, że możliwe jest zbudowanie lokalnego menedżera haseł w architekturze zerowej wiedzy, którego poufność i integralność danych nie zależą od zaufania do serwera, a poprawność warstwy kryptograficznej podlega weryfikacji automatycznej. Potwierdzono również, że rozwiązanie takie można wyposażyć w zaawansowane mechanizmy ochrony, w tym fałszywy skarbiec otwierany pod przymusem o jednakowym czasie odblokowania, przy jednoczesnym, uczciwym nazwaniu i udokumentowaniu ich ograniczeń wynikających z natury platformy webowej.

Za szczególny wkład pracy należy uznać trzy elementy. Po pierwsze, połączenie weryfikowalnej poprawności kryptograficznej z pełną przejrzystością implementacji, w której każda decyzja jest udokumentowana, a jej skutki możliwe do prześledzenia na poziomie kodu źródłowego. Po drugie, zaprojektowanie i realizację zaawansowanych mechanizmów ochrony wykraczających poza standard typowego menedżera haseł. Po trzecie, konsekwentne, jawne nazywanie ograniczeń, które potraktowano jako pełnoprawny przedmiot analizy, a nie jako element do przemilczenia. To właśnie ta rzetelność, obok poprawności

kryptografii, stanowi o wiarygodności narzędzia bezpieczeństwa.

Praca nie wyczerpuje tematu, lecz tworzy solidny fundament do dalszego rozwoju. Wskazane kierunki, synchronizacja wielu urządzeń, rozszerzenie przeglądarki, ukryty wolumin, klucze sprzętowe oraz formalny audyt bezpieczeństwa, wytyczają drogę dojrzewania rozwiązania od akademickiego prototypu ku narzędziu gotowemu do zastosowań praktycznych, przy niezmiennym zachowaniu nadrzędnej zasady: hasło główne nigdy nie opuszcza urządzenia użytkownika.

BIBLIOGRAFIA

[1] AgileBits Inc., *1Password Security Design* (white paper), https://1password.com/security (dostęp: 20.06.2026).

[2] Biryukov A., Dinu D., Khovratovich D., *Argon2: the Memory-Hard Function for Password Hashing and Other Applications*, Password Hashing Competition, 2017, https://www.password-hashing.net (dostęp: 20.06.2026).

[3] Biryukov A., Dinu D., Khovratovich D., Josefsson S., *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications* (RFC 9106), Internet Engineering Task Force, 2021, https://www.rfc-editor.org/rfc/rfc9106 (dostęp: 20.06.2026).

[4] Bitwarden Inc., *Bitwarden Security Whitepaper*, https://bitwarden.com/help/bitwarden-security-white-paper (dostęp: 20.06.2026).

[5] Bonneau J., Herley C., van Oorschot P., Stajano F., *The Quest to Replace Passwords: A Framework for Comparative Evaluation of Web Authentication Schemes*, „IEEE Symposium on Security and Privacy" 2012, s. 553–567, https://ieeexplore.ieee.org (dostęp: 20.06.2026).

[6] Dworkin M., *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC* (NIST SP 800-38D), National Institute of Standards and Technology, Gaithersburg 2007, https://csrc.nist.gov (dostęp: 20.06.2026).

[7] ENISA, *ENISA Threat Landscape 2023*, European Union Agency for Cybersecurity, 2023, https://www.enisa.europa.eu/publications (dostęp: 20.06.2026).

[8] FastAPI, *FastAPI Documentation*, https://fastapi.tiangolo.com (dostęp: 20.06.2026).

[9] Ferguson N., Schneier B., Kohno T., *Cryptography Engineering: Design Principles and Practical Applications*, Wiley, Indianapolis 2010, https://www.wiley.com (dostęp: 20.06.2026).

[10] Fielding R., Nottingham M., Reschke J., *HTTP Semantics* (RFC 9110), Internet Engineering Task Force, 2022, https://www.rfc-editor.org/rfc/rfc9110 (dostęp: 20.06.2026).

[11] Grassi P., Garcia M., Fenton J., *Digital Identity Guidelines: Authentication and Lifecycle Management* (NIST SP 800-63B), National Institute of Standards and Technology, Gaithersburg 2017, https://pages.nist.gov/800-63-3 (dostęp: 20.06.2026).

[12] Hunt T., *Have I Been Pwned: Check if your email has been compromised in a data breach*, https://haveibeenpwned.com (dostęp: 20.06.2026).

[13] Katz J., Lindell Y., *Introduction to Modern Cryptography*, wyd. 2, CRC Press, Boca Raton 2014, https://www.routledge.com (dostęp: 20.06.2026).

[14] KeePass, *KeePass Password Safe, Help Center*, https://keepass.info/help (dostęp: 20.06.2026).

[15] LastPass, *Notice of Recent Security Incident*, LastPass US LP, 2022, https://blog.lastpass.com (dostęp: 20.06.2026).

[16] Li Z., He W., Akhawe D., Song D., *The Emperor's New Password Manager: Security Analysis of Web-based Password Managers*, „USENIX Security Symposium" 2014, s. 465–479, https://www.usenix.org (dostęp: 20.06.2026).

[17] Moriarty K., Kaliski B., Rusch A., *PKCS #5: Password-Based Cryptography Specification Version 2.1* (RFC 8018), Internet Engineering Task Force, 2017, https://www.rfc-editor.org/rfc/rfc8018 (dostęp: 20.06.2026).

[18] Mozilla, *Web Crypto API, MDN Web Docs*, Mozilla Foundation, https://developer.mozilla.org/docs/Web/API/Web_Crypto_API (dostęp: 20.06.2026).

[19] Mozilla, *Content Security Policy (CSP), MDN Web Docs*, Mozilla Foundation, https://developer.mozilla.org/docs/Web/HTTP/CSP (dostęp: 20.06.2026).

[20] M'Raihi D., Machani S., Pei M., Rydell J., *TOTP: Time-Based One-Time Password Algorithm* (RFC 6238), Internet Engineering Task Force, 2011, https://www.rfc-editor.org/rfc/rfc6238 (dostęp: 20.06.2026).

[21] OWASP Foundation, *Credential Stuffing Prevention Cheat Sheet*, https://cheatsheetseries.owasp.org (dostęp: 20.06.2026).

[22] OWASP Foundation, *Password Storage Cheat Sheet*, https://cheatsheetseries.owasp.org (dostęp: 20.06.2026).

[23] OWASP Foundation, *OWASP Top Ten*, OWASP, 2021, https://owasp.org/www-project-top-ten (dostęp: 20.06.2026).

[24] Pearman S., Thomas J., Naeini P. i in., *Why People (Don't) Use Password Managers Effectively*, „Symposium on Usable Privacy and Security (SOUPS)" 2019, https://www.usenix.org (dostęp: 20.06.2026).

[25] React, *React Documentation*, Meta Open Source, https://react.dev (dostęp: 20.06.2026).

[26] Schaad J., Housley R., *Advanced Encryption Standard (AES) Key Wrap Algorithm* (RFC 3394), Internet Engineering Task Force, 2002, https://www.rfc-editor.org/rfc/rfc3394 (dostęp: 20.06.2026).

[27] Stallings W., *Cryptography and Network Security: Principles and Practice*, wyd. 7, Pearson, Boston 2017, https://www.pearson.com (dostęp: 20.06.2026).

[28] TypeScript, *TypeScript Handbook*, Microsoft, https://www.typescriptlang.org/docs (dostęp: 20.06.2026).

[29] Verizon, *Data Breach Investigations Report (DBIR)*, Verizon Business, https://www.verizon.com/business/resources/reports/dbir (dostęp: 20.06.2026).

[30] W3C, *Content Security Policy Level 3*, World Wide Web Consortium, https://www.w3.org/TR/CSP3 (dostęp: 20.06.2026).

[31] W3C, *Web Authentication: An API for Accessing Public Key Credentials (WebAuthn) Level 2*, World Wide Web Consortium, 2021, https://www.w3.org/TR/webauthn-2 (dostęp: 20.06.2026).

SPIS RYSUNKÓW

3.1 Czas wyprowadzenia klucza Argon2id w funkcji kosztu pamięciowego oraz liczby iteracji t (urządzenie referencyjne, $p = 1$) 22
3.2 Model KEK/DEK, klucz wyprowadzony z hasła (KEK) opakowuje losowy klucz danych (DEK), który szyfruje skarbiec 25

4.1 Ogólny podział systemu w modelu zerowej wiedzy 27
4.2 Przepływ odblokowania skarbca, po jednym pobraniu pakietu cała reszta odbywa się po stronie klienta; hasło główne nigdy nie trafia do serwera . . 33

5.1 Dwuslotowa konstrukcja fałszywego skarbca, oba sloty szyfrowane niezależnie, otwierane różnymi hasłami; KEK obu slotów wyprowadzane są zawsze, co zapewnia jednakowy czas odblokowania 36
5.2 Narastające wykładniczo opóźnienie po kolejnych nieudanych próbach odblokowania, pierwsze trzy próby bez kary, następnie podwajanie aż do limitu 300 s 40

7.1 Zrzut ekranu z wykonania zestawu testów warstwy klienckiej: 20 plików testowych, 124 przypadki, wszystkie zaliczone 51
7.2 Zrzut ekranu z wykonania zestawu testów warstwy serwerowej: 19 przypadków, wszystkie zaliczone 53
7.3 Zrzut ekranu z pomiaru pokrycia kodu logicznego jądra aplikacji 55
7.4 Zrzut ekranu z wykonywalnego dowodu, że hasło główne nie opuszcza przeglądarki: zero żądań zawierających hasło spośród 152 nagranych . . . 56
7.5 Zrzut ekranu z wierszami podsumowującymi wszystkie 20 scenariuszy całościowych, brak porażek 57

D.1 Ekran tworzenia skarbca w stanie początkowym: miernik entropii wskazuje wartość zerową i ocenę „słabe”, przy której bramka nie dopuszcza utworzenia skarbca 78
D.2 Miernik entropii hasła głównego wraz z oszacowaniem siły w bitach . . . 79
D.3 Ekran blokady, na który aplikacja przechodzi po zablokowaniu sesji, porzucając klucz danych i odszyfrowaną treść skarbca 79
D.4 Odblokowany skarbiec w układzie dwupanelowym: lista wpisów oraz panel szczegółów 80

D.5 Audyt zdrowia haseł: liczniki haseł słabych, zbyt krótkich, starych oraz powtórzonych wraz z listą wykrytych wpisów 80
D.6 Generator haseł oparty na kryptograficznie bezpiecznym generatorze liczb losowych 81
D.7 Ekran ustawień po odblokowaniu skarbca hasłem przynęty: sekcja zarządzania fałszywym skarbcem jest ukryta, dzięki czemu interfejs nie zdradza istnienia drugiego slotu 81
D.8 Rotacja klucza danych (DEK) wykonywana bez zmiany hasła głównego; widoczne parametry KDF (256 MiB, $t = 2$) dobrała kalibracja w środowisku testowym i różnią się one od parametrów urządzenia referencyjnego z tabeli 3.1, co ilustruje omówioną w rozdziale 3 zależność siły ochrony od sprzętu 82
D.9 Świadome fokusu czyszczenie schowka wraz ze wspólnym licznikiem pozostałego czasu 82
D.10 Optymistyczna kontrola współbieżności: druga karta otrzymuje informację o konflikcie zamiast cicho nadpisać zmiany 83

SPIS TABEL

1.1 Modele przechowywania skarbca a wymagane zaufanie 9

2.1 Porównanie wybranych menedżerów haseł 13
2.2 Model zagrożeń projektowanego menedżera haseł 15
2.3 Wymagania bezpieczeństwa projektowanego systemu 17
2.4 Wymagania funkcjonalne projektowanego systemu 18

3.1 Czas wyprowadzenia klucza funkcją Argon2id na urządzeniu referencyjnym ($p = 1$) 21
3.2 Kontrakt kryptograficzny projektu 26

4.1 Podział odpowiedzialności pomiędzy klienta a serwer 28
4.2 Pola pakietu danych 29
4.3 Wybrane nagłówki bezpieczeństwa ustawiane przez serwer 32

5.1 Progi siły hasła głównego i reakcja bramki 38
5.2 Zestawienie zaawansowanych mechanizmów ochrony 42

6.1 Co może, a co nie może trafić do pamięci trwałej przeglądarki 45

7.1 Stos technologiczny projektu 47
7.2 Poziomy strategii testów 49
7.3 Środowisko wykonania testów (urządzenie referencyjne) 51
7.4 Wyniki testów warstwy klienckiej według obszarów 52
7.5 Wyniki testów warstwy serwerowej 54
7.6 Pokrycie kodu logicznego jądra aplikacji 55
7.7 Scenariusze testów całościowych i ich wyniki 58
7.8 Zbiorcze wyniki wykonania wszystkich poziomów testów 59

8.1 Zestawienie dowodów weryfikacji 61
8.2 Kierunki dalszego rozwoju 63

A.1 Słownik terminów i skrótów 74

SPIS LISTINGÓW

3.1 Metadane KDF zapisywane w pakiecie danych (TypeScript) 23
3.2 Szyfrowanie danych ze świeżym wektorem inicjującym (TypeScript) . . . 24
3.3 Zmiana hasła jako ponowne opakowanie klucza danych (TypeScript) . . . 25
4.1 Typ pojedynczego slotu pakietu danych (TypeScript) 29
4.2 Schemat pakietu z odrzucaniem obcych pól (Python, Pydantic) 30
4.3 Optymistyczna kontrola współbieżności (Python) 31
5.1 Równoległe wyprowadzenie kluczy obu slotów dla jednakowego czasu odblokowania (TypeScript) 35
5.2 Strażnik różności haseł przy tworzeniu przynęty (TypeScript) 35
5.3 Szacowanie entropii hasła w bitach (TypeScript) 37
5.4 Wyzwalacz bezczynności porzucający sesję (TypeScript) 39
5.5 Wykładnicze opóźnienie nieudanych prób (TypeScript) 39
6.1 Polityka bezpieczeństwa treści serwowana przez serwer 44
6.2 Granica błędów porzucająca klucz danych przy awarii renderu (TypeScript) 46
7.1 Test znanej odpowiedzi dla PBKDF2 względem wektora standardowego (TypeScript) 50
B.1 Poglądowa struktura pakietu danych z opcjonalnym slotem przynęty . . . 76

Załączniki

Dodatek A

SŁOWNIK TERMINÓW I SKRÓTÓW

W dodatku zestawiono najważniejsze terminy i skróty użyte w pracy wraz z ich zwięzłym objaśnieniem (tabela A.1).

Tab. A.1: Słownik terminów i skrótów

Termin / skrót	Objaśnienie
Zerowa wiedza	(ang. *zero-knowledge*) model, w którym serwer nigdy nie poznaje hasła ani kluczy, a całość kryptografii odbywa się po stronie klienta
KDF	(ang. *Key Derivation Function*) funkcja wyprowadzania klucza z hasła, celowo kosztowna obliczeniowo i pamięciowo
Argon2id	twarda pamięciowo funkcja KDF (RFC 9106) [3], główna funkcja wyprowadzania klucza w projekcie
PBKDF2	funkcja KDF oparta na iterowanym HMAC; w projekcie udokumentowany wariant zapasowy
AEAD	(ang. *Authenticated Encryption with Associated Data*) szyfrowanie zapewniające jednocześnie poufność i integralność
AES-256-GCM	szyfr blokowy AES w trybie Galois/Counter Mode; szyfruje treść skarbca
AES-KW	(ang. *AES Key Wrap*, RFC 3394) [26] algorytm opakowania kluczy, uwierzytelniony i bez wektora inicjującego
KEK	(ang. *Key Encryption Key*) klucz opakowujący wyprowadzony z hasła
DEK	(ang. *Data Encryption Key*) losowy klucz danych szyfrujący skarbiec

ciąg dalszy na następnej stronie

Tab. A.1, ciąg dalszy

Termin / skrót	**Objaśnienie**
IV	(ang. *Initialization Vector*) wektor inicjujący; w AES-GCM losowy i świeży dla każdej operacji
Sól	losowa wartość dołączana do hasła przed wyprowadzeniem klucza; nie jest sekretem
Pakiet danych	(ang. *envelope*) nieprzejrzysta struktura przechowywana przez serwer (metadane KDF, opakowany DEK, IV, szyfrotekst)
Entropia	miara nieprzewidywalności hasła wyrażona w bitach
CSPRNG	(ang. *Cryptographically Secure PRNG*) kryptograficznie bezpieczny generator liczb losowych
CSP	(ang. *Content Security Policy*) [30] polityka bezpieczeństwa treści ograniczająca źródła zasobów
XSS	(ang. *Cross-Site Scripting*) atak polegający na wstrzyknięciu i wykonaniu obcego skryptu
TOTP	(ang. *Time-Based One-Time Password*, RFC 6238) [20] algorytm kodów jednorazowych zależnych od czasu
ETag / If-Match	nagłówki HTTP [10] realizujące optymistyczną kontrolę współbieżności
Fałszywy skarbiec	opcjonalny drugi slot pakietu otwierany innym hasłem, ochrona przed wymuszeniem
Rotacja DEK	wymiana klucza danych i ponowne zaszyfrowanie skarbca bez zmiany hasła

Dodatek B

PRZYKŁADOWA STRUKTURA PAKIETU DANYCH

Listing B.1 przedstawia poglądową strukturę pakietu danych w formacie JSON, dokładnie tak, jak przechowuje go serwer, jako nieprzejrzysty obiekt. Wartości binarne (sól, opakowany klucz danych, wektor inicjujący, szyfrotekst) zakodowano w base64 i skrócono. Opcjonalne pole `przyneta` zawiera niezależnie zaszyfrowany drugi slot (fałszywy skarbiec).

```
{
  "wersja": 1,
  "kdf": {
    "nazwa": "argon2id",
    "sol": "b3Jq...",
    "m": 131072, "t": 3, "p": 1
  },
  "dekOpakowany": "S2VL...",
  "iv": "aXZpdg...",
  "szyfrotekst": "Y2lwaGVy...",
  "przyneta": {
    "kdf": { "nazwa": "argon2id", "sol": "c2FsdA...", "m": 131072, "
       t": 3, "p": 1 },
    "dekOpakowany": "ZGVr...",
    "iv": "aXYy...",
    "szyfrotekst": "ZGVjb3k..."
  }
}
```

Listing B.1: Poglądowa struktura pakietu danych z opcjonalnym slotem przynęty
Źródło: opracowanie własne

Należy podkreślić, że dla serwera cała powyższa struktura jest jedynie ciągiem bajtów: nie interpretuje on metadanych KDF, nie rozróżnia slotu głównego od slotu przynęty i nie jest w stanie odszyfrować żadnego z pól. Pełną kontrolę nad treścią sprawuje wyłącznie klient dysponujący hasłem.

Dodatek C

STRUKTURA KATALOGÓW PROJEKTU

Poniższe zestawienie przedstawia uproszczoną strukturę katalogów repozytorium (pominięto katalogi narzędziowe: `node_modules`, `.venv`, `dist`). Podział odzwierciedla architekturę zerowej wiedzy: niezależne warstwy klienta (`frontend`) i serwera (`backend`) oraz wydzielony rdzeń kryptograficzny.

```
Password_Manager/
|- backend/                  serwer FastAPI (magazyn pakietu danych)
|  |- app/
|  |  |- main.py             endpointy GET/PUT /skarbiec
|  |  |- schematy.py         walidacja schematu (extra="forbid")
|  |  |- security.py         nagłówki bezpieczeństwa (CSP, nosniff)
|  |  '- storage.py          utrwalanie + kontrola rewizji (ETag)
|  '- requirements.txt       zależności Pythona
|- frontend/                 aplikacja React + TypeScript
|  |- src/
|  |  |- crypto/             rdzeń kryptograficzny (model KEK/DEK)
|  |  |  |- argon2.ts        Argon2id przez hash-wasm
|  |  |  |- pbkdf2.ts        wariant zapasowy PBKDF2
|  |  |  |- skarbiec.ts      klucze, sloty, rotacja DEK, przynęta
|  |  |  '- typy.ts          format pakietu danych (typy)
|  |  |- api/                komunikacja z serwerem (If-Match)
|  |  |- audyt/              audyt zdrowia haseł
|  |  |- ekran/              ekran blokady, miernik entropii
|  |  |- skarbiec/           widok skarbca, generator, TOTP, kosz
|  |  |- powloka/            paleta poleceń, granica błędów
|  |  |- entropia.ts         szacowanie entropii hasła
|  |  |- blokadaProb.ts      dławienie nieudanych prób
|  |  |- schowek.ts          czyszczenie schowka świadome fokusu
|  |  '- bezczynnosc.ts      auto-blokada po bezczynności
|  |- bench/                 pomiar wzorcowy Argon2id
|  |- package.json           zależności i skrypty npm
|  '- vite.config.ts         konfiguracja budowania (Vite)
|- .github/workflows/ci.yml  CI: testy, próg pokrycia, audyt zależności
'- README.md                 opis projektu i instrukcja uruchomienia
```

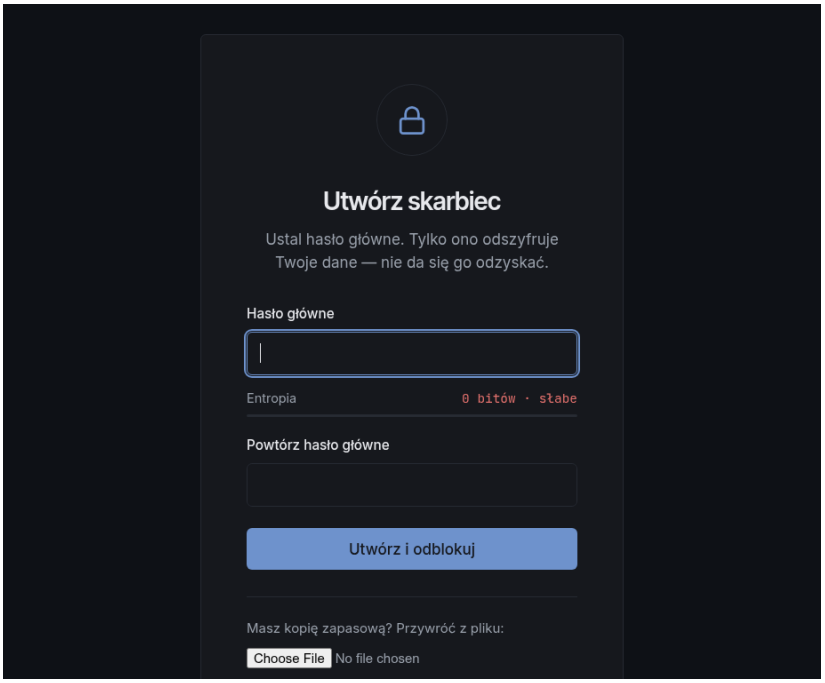
Źródło: opracowanie własne

Taki układ wymusza rozdział odpowiedzialności opisany w rozdziale 4: katalog `backend` nie zawiera żadnej logiki kryptograficznej, a cała kryptografia skupiona jest w `frontend/src/crypto`.

Dodatek D

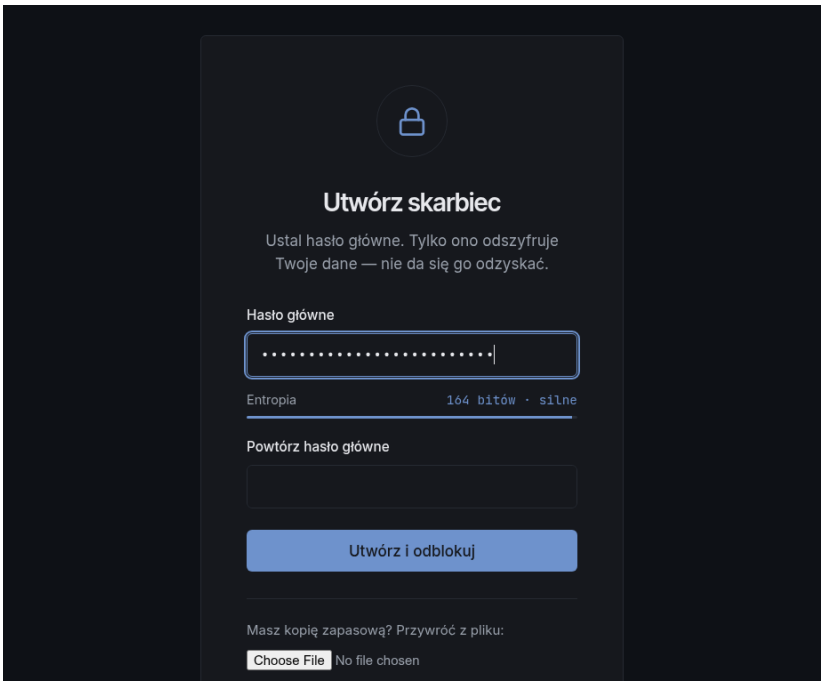
ZRZUTY EKRANU Z DZIAŁANIA APLIKACJI

Zrzuty zamieszczone w niniejszym dodatku pochodzą z automatycznych przebiegów testów całościowych opisanych w podrozdziale 7.4.4: każdy scenariusz, poza sprawdzeniem asercji, zapisuje obraz badanego widoku. Nie są to zatem ilustracje przygotowane ręcznie, lecz materiał wytworzony w trakcie weryfikacji, dokumentujący rzeczywisty stan aplikacji w momencie wykonania testu.

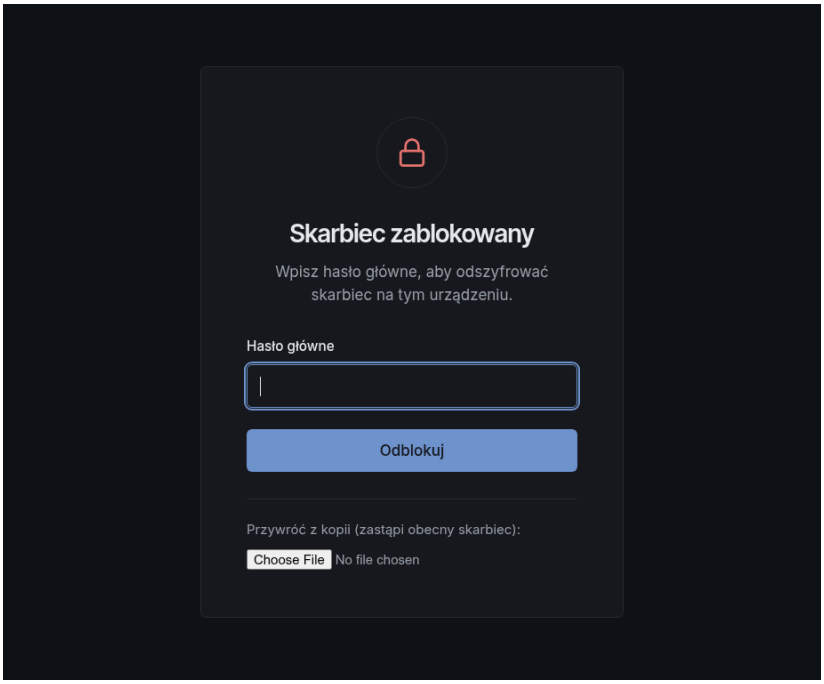


Rys. D.1: Ekran tworzenia skarbca w stanie początkowym: miernik entropii wskazuje wartość zerową i ocenę „słabe”, przy której bramka nie dopuszcza utworzenia skarbca

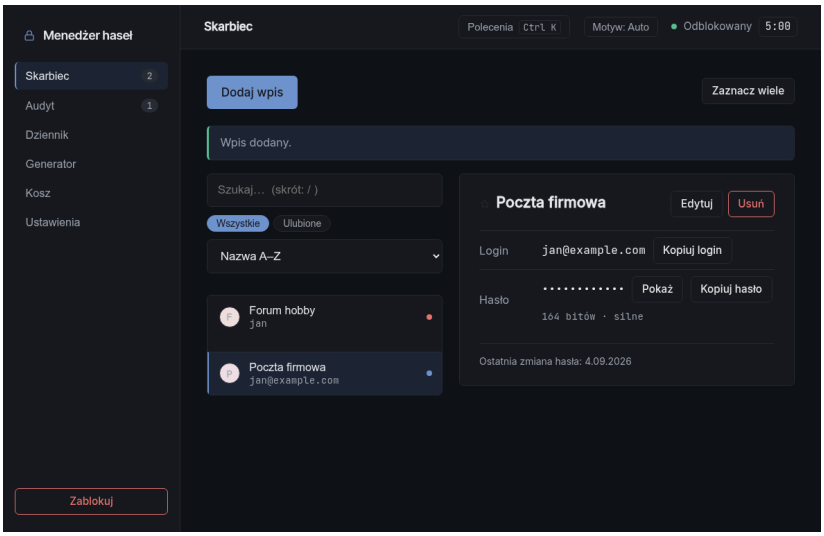
Źródło: opracowanie własne (zrzut ekranu z przebiegu testów)



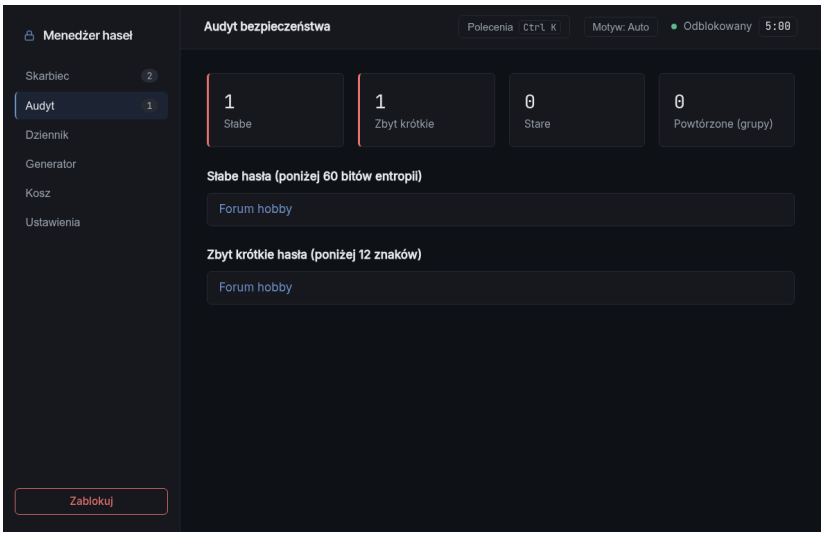
Rys. D.2: Miernik entropii hasła głównego wraz z oszacowaniem siły w bitach
Źródło: opracowanie własne (zrzut ekranu z przebiegu testów)



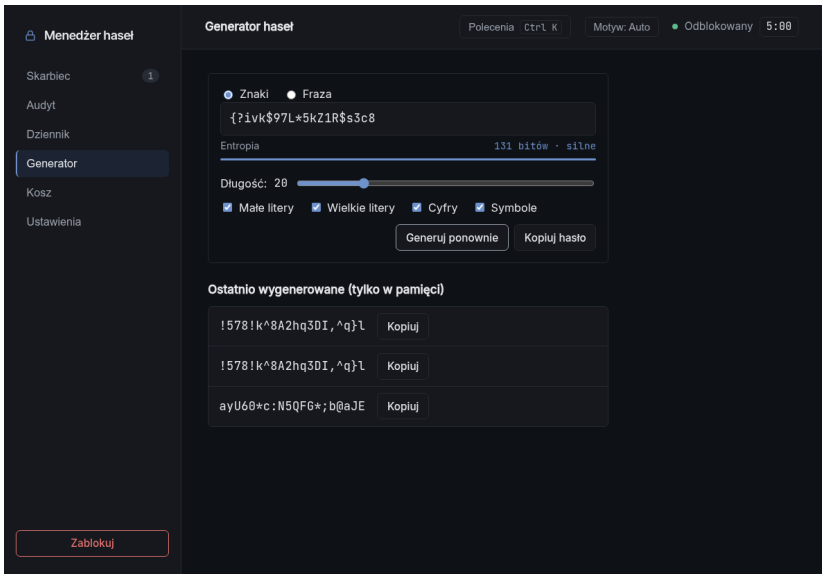
Rys. D.3: Ekran blokady, na który aplikacja przechodzi po zablokowaniu sesji, porzucając klucz danych i odszyfrowaną treść skarbca
Źródło: opracowanie własne (zrzut ekranu z przebiegu testów)



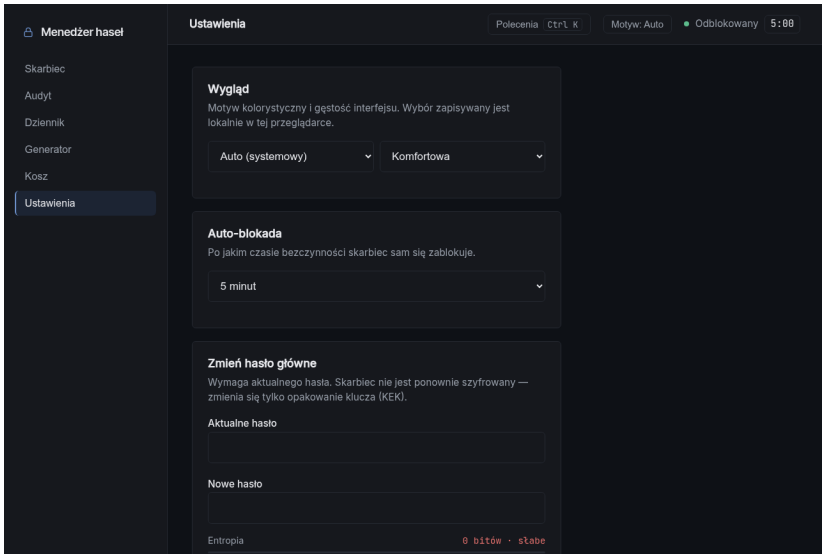
Rys. D.4: Odblokowany skarbiec w układzie dwupanelowym: lista wpisów oraz panel szczegółów
Źródło: opracowanie własne (zrzut ekranu z przebiegu testów)



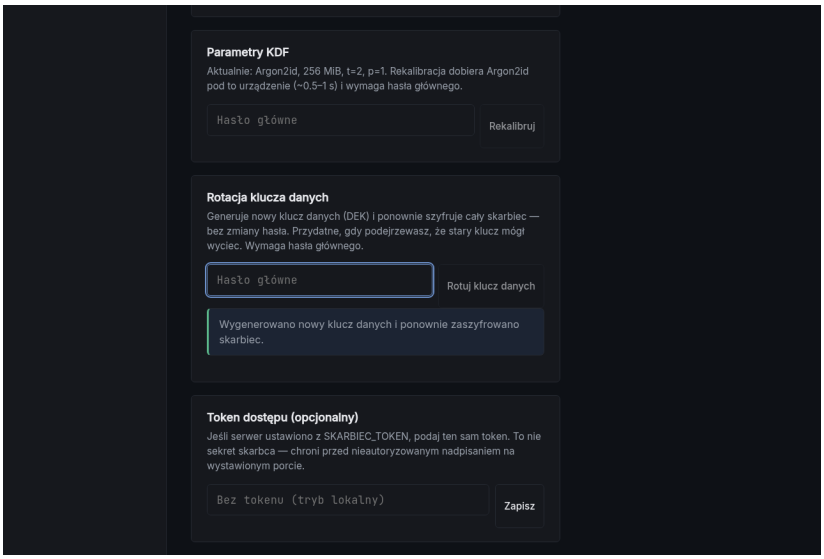
Rys. D.5: Audyt zdrowia haseł: liczniki haseł słabych, zbyt krótkich, starych oraz powtórzonych wraz z listą wykrytych wpisów
Źródło: opracowanie własne (zrzut ekranu z przebiegu testów)



Rys. D.6: Generator haseł oparty na kryptograficznie bezpiecznym generatorze liczb losowych
Źródło: opracowanie własne (zrzut ekranu z przebiegu testów)

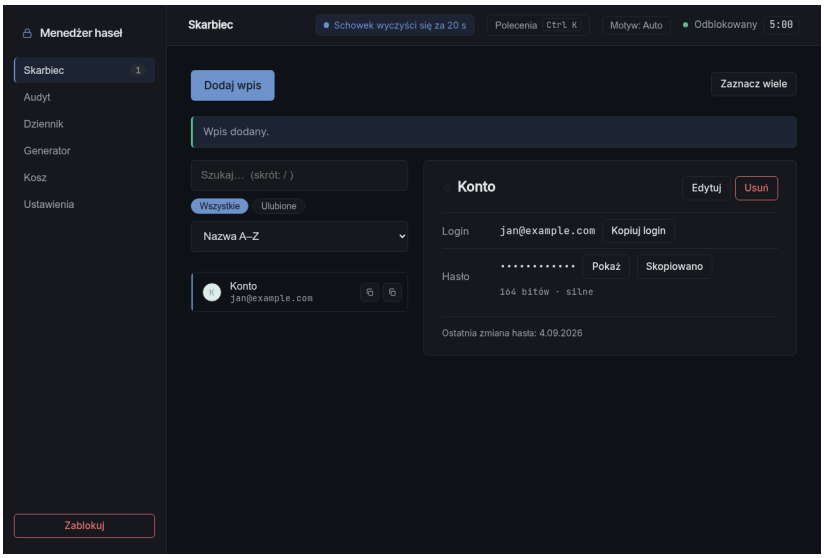


Rys. D.7: Ekran ustawień po odblokowaniu skarbca hasłem przynęty: sekcja zarządzania fałszywym skarbcem jest ukryta, dzięki czemu interfejs nie zdradza istnienia drugiego slotu
Źródło: opracowanie własne (zrzut ekranu z przebiegu testów)



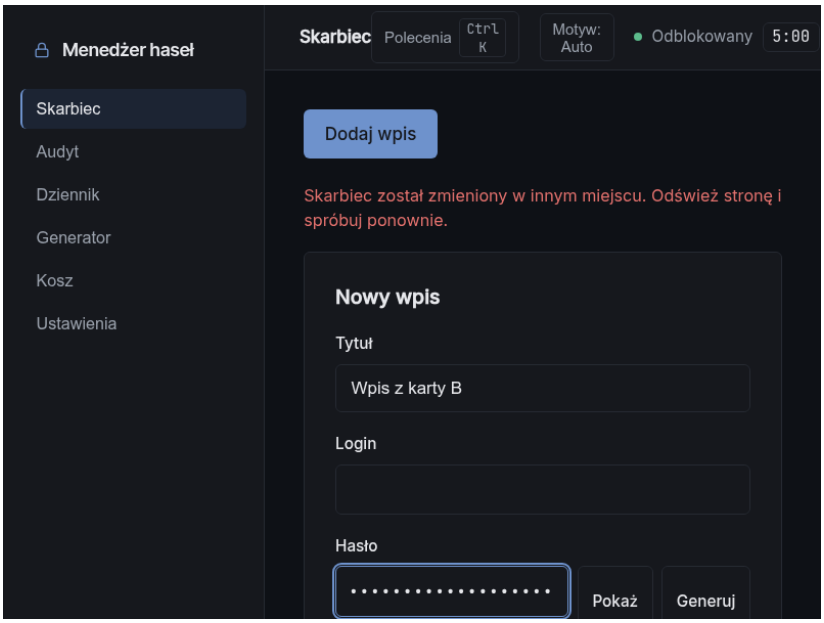
Rys. D.8: Rotacja klucza danych (DEK) wykonywana bez zmiany hasła głównego; widoczne parametry KDF (256 MiB, $t = 2$) dobrała kalibracja w środowisku testowym i różnią się one od parametrów urządzenia referencyjnego z tabeli 3.1, co ilustruje omówioną w rozdziale 3 zależność siły ochrony od sprzętu

Źródło: opracowanie własne (zrzut ekranu z przebiegu testów)



Rys. D.9: Świadome fokusu czyszczenie schowka wraz ze wspólnym licznikiem pozostałego czasu

Źródło: opracowanie własne (zrzut ekranu z przebiegu testów)



Rys. D.10: Optymistyczna kontrola współbieżności: druga karta otrzymuje informację o konflikcie zamiast cicho nadpisać zmiany

Źródło: opracowanie własne (zrzut ekranu z przebiegu testów)